

分类号: TP391

密级: 公开

学校代码: 11065

学号: 2019025986

青岛大学

专业硕士学位论文

(统招全日制)

高分辨率海洋标量数据动态可视化方法研究

作者姓名	张广宇
指导教师	张小垒 讲师
专业领域	空间数据可视化
培养单位	计算机科学技术学院
答辩日期	2022 年 6 月 4 日

摘要

近年随着显示设备的飞速发展,高分辨率数据可视化大屏展示的需求愈加增多,海洋观测数据可视化系统也逐渐倾向于使用高分辨率大屏进行交互展示。同时,海洋观测技术的不断发展使得观测数据的时空分辨率不断提高,为高分辨率数据可视化提供了数据支持。传统的全球视域海洋数据可视化主要采用 LOD 算法:在全球视域下渲染低分辨率数据,在局部视域下渲染高分率数据。由于低分辨率数据可视化在大屏上展示效果比较模糊,因此,该算法不适用于目前大屏环境中全球视域的可视化展示。所以,如何实时处理和渲染不同数据源的高分辨率海洋标量数据,提高数据渲染实时性,实现全球视角下全域、高效的三维实时动态可视化;解放屏幕大小,满足大屏可视化的需求,提供更为清晰、准确的海洋变化信息,是本文的主要研究目标。

基于这种目标,针对当前研究现状及现有研究的不足,本文提出一种基于 web 端的海洋标量场数据动态可视化方法,设计实现了一个面向高分辨率大屏互动展示的海洋标量场动态可视化系统。系统利用人眼色差识别的研究结果确定时变阈值,筛选变化数据,从而减少时变渲染数据量;采用分布式框架,多台服务器按照本文所设计的数据可视化方法进行数据预处理和数据文件更新,组织不同经纬度范围内的时变数据,利用 WebSocket 协议实现数据实时推送;借助 GPU 实现数据与渲染命令按需更新,提升渲染速度;利用 MQTT 协议实现移动端与大屏之间的通信连接,借助移动端实现大屏可视化的交互操作。

最后,本文借助多种类型的海洋标量数据测试所开发系统的渲染性能。经过测试,本文提出的数据可视化方法在保证可视化效果的前提下缩短了时变渲染时间,增强了数据渲染实时性,可以满足实时性较高的应用场景需求,帮助用户快速掌握数据变化规律。基于此方法所设计的系统解决了数据分辨率对于屏幕尺寸的限制,避免了前后端频繁的数据交互,从而适应不同的网络环境。其提供的多源数据可视化帮助用户掌握不同种类数据的变化规律,为之后的科研活动、军事活动提供信息支持,助力国家海洋信息化发展,帮助国家科学开发、利用海洋资源。

关键词: 高分辨率, 实时动态可视化, 大屏可视化, 人眼色差识别,

GPU 渲染

Abstract

In recent years, with the rapid development of display equipment, the demand for large screen display of high-resolution data visualization is increasing, and the ocean observation data visualization system gradually tends to use large screen for interactive display. At the same time, with the continuous development of ocean observation technology, the spatial-temporal resolution of observation data is continuously improved, which provides data support for high-resolution data visualization. The traditional global view ocean data visualization mainly adopts LOD algorithm: rendering low-resolution data in the global view and high-resolution data in the local view. Due to the fuzzy display effect of low resolution data visualization on the large screen, the algorithm is not suitable for the visual display of global view in the current large screen environment. Therefore, the real-time processing and rendering of high-resolution ocean scalar data from different data sources can improve the real-time performance of data rendering; Realize global and efficient 3D real-time dynamic visualization from a global perspective; Liberate the screen size, meet the needs of large screen visualization, and provide clearer and accurate ocean change information. These are the main research objectives of this paper.

Based on this goal, aiming at the current research status and the shortcomings of existing research, this paper proposes a dynamic visualization method of marine scalar field data based on Web. A dynamic visualization system of ocean scalar field for high-resolution large screen interactive display is designed and implemented. The system uses the research results of human eye color difference recognition to determine the time-varying threshold and screen the changing data, so as to reduce the amount of time-varying rendering data. With the help of GPU, the data and rendering commands can be updated on demand to improve the rendering speed. Using the distributed framework, multiple servers carry out data preprocessing and data file updating according to the data visualization method designed in this paper, organize time-varying data in different longitude and latitude ranges, and use WebSocket protocol to realize real-time data push. MQTT protocol is used to realize the communication connection between mobile terminal and large screen, and the interactive operation of large screen visualization is realized with the help of mobile terminal.

Finally, this paper tests the rendering performance of the developed system with the help of various types of ocean scalar data. After testing, the data visualization

method proposed in this paper shortens the time-varying rendering time and enhances the real-time performance of data rendering on the premise of ensuring the visualization effect. It can meet the needs of high real-time application scenarios and help users quickly grasp the law of data change. The system designed based on this method solves the limitation of data resolution on screen size. It avoids the frequent data interaction between the front and back ends, so as to adapt to different network environments. The multi-source data visualization provided by it helps users master the change law of different types of data, provides information support for subsequent scientific research and military activities, helps the development of national marine informatization, and helps the country scientifically develop and utilize marine resources.

Keywords: high resolution, real-time dynamic visualization, large screen visualization, human eye color difference recognition, GPU rendering

目录

第一章 引言	1
1.1 课题研究背景与意义	1
1.2 国内外研究现状	2
1.3 本文主要研究内容与论文组织结构	3
第二章 相关技术介绍	4
2.1 MQTT 通信服务	5
2.1.1 MQTT 通信协议	5
2.1.2 MQTT 通信协议原理	5
2.1.3 MQTT 报文结构	7
2.2 WebSocket 通信服务	9
2.2.1 WebSocket 通信协议	9
2.2.2 WebSocket 通信协议建立	9
2.3 WebGL 渲染	11
2.4 Cesium 框架	13
2.5 本章小结	14
第三章 基于色差阈值的时变数据可视化方法	16
3.1 色差时变阈值	16
3.1.1 CIEDE2000 色差公式	16
3.1.2 时变阈值实验设计	17
3.1.3 时变阈值选择	18
3.2 时变数据可视化	20
3.2.1 时变数据结构	20
3.2.2 数据预处理	21
3.2.3 数据可视化	22
3.3 本章小结	24
第四章 系统设计与实现	26
4.1 系统体系架构	26
4.1.1 系统框架	26
4.1.2 系统层级	27
4.2 系统模块设计	29
4.2.1 数据通信模块设计	29
4.2.2 数据预处理模块设计	34
4.2.2.1 Node.JS	35
4.2.2.2 Node.JS Express 框架	37
4.2.2.3 具体流程设计	38
4.2.3 数据渲染模块	40
4.3 系统功能实现	42
4.3.1 图层切换	43
4.3.2 时变图层渲染	45
4.3.3 数据查询	49
4.3.4 自定义范围划定	50

4.3.5 图表绘制	51
4.4 本章小结	52
第五章 系统性能测试	53
5.1 不同分辨率渲染图层效果对比	53
5.2 数据通信性能对比	54
5.3 数据渲染性能对比	55
5.4 本章小结	60
第六章 总结与展望	61
参考文献	62
致谢	66
攻读学位期间的研究成果	67
学位论文独创性声明	68
学位论文知识产权权属声明	68

第一章 引言

1.1 课题研究背景与意义

海洋蕴含着包括水资源、生物资源以及矿产资源在内的多种自然资源^[1]，覆盖全球表面积的 71%，地球 97% 的水资源都来源于海洋，为生物提供了 99% 的生存空间。人类的可持续发展，都依赖于健康的海洋^[2]。

海洋中的生物为人类提供充足的食物来源，其多样性不仅丰富了地球物种，而且使整个海洋生态链得以循环，维持海洋生态系统的平衡^[3]。人类通过研究海洋生物的生命进程、演化等活动，为分析海洋气候变化、研究其他生物物种来源、探索地球起源等科研活动提供了重要参考。海洋中石油、天然气等矿产资源为人类日常生活提供不可或缺的能源供应。此外，海洋温度、洋流、季风等自然气候时刻影响着海洋环境，由于气候变化导致的台风、海啸等也影响着人类的生产生活^[4]。

为了可以更加直观地观察到不同种类海洋信息变化，更准确地掌握海洋信息变化规律，海洋数据可视化技术^[5]应运而生。随着时间推移，可视化技术逐渐成熟，其将枯燥的数据借助图形化技术加以展示，直观表达其中蕴含的信息，被广泛应用于海洋渔业、交通、环境监测等领域^[6]。由于海洋探测技术的不断进步，数据搜集设备的日益完善，收集到的海洋数据的分辨率越来越高^[7]，可以提供更加精准、详细的海洋信息。硬件技术的发展使得屏幕制作技术有了跨越式的提高^[8]，屏幕尺寸越来越大，可以满足高分辨率数据全域动态可视化展示的需求，大屏数据可视化^[9-10]开始兴起。可视化技术面对分辨率日益提高的海洋数据和尺寸日益增大的屏幕，在向用户展现更加准确信息的同时，也面临着渲染数据量增多，动态可视化难度增大的挑战。

党的十八大报告中明确提出：要不断增强开发海洋的能力，进而维护国家海洋权益，建设海洋强国^[11]。由此可知，积极探索海洋，科学开采海洋资源，坚决维护海洋主权，强军强国，将是我国未来发展的关键方向。海洋信息化是实现国家信息化的关键一步，为发展国家海洋科研事业提供充足动力，在我国海洋事业发展中发挥着关键作用^[12]。预测‘杜鹃’、‘烟花’等台风的运动方向，提醒相关地区提前做好准备；蛟龙号及其他先进潜水艇不断创造下潜纪录，逐渐揭开海洋深处的面纱；国产航母下水，国产 055 型驱逐舰成功入编航母编队，保卫国家海洋主权。这些成就彰显了我国在民生、军事上信息化建设的伟大进步。

本世纪被称为是海洋的世纪，海洋攸关国家安全和长远发展，全球大多数海洋国家将海洋权益分属为国家核心权益，积极主动推进海洋经济政策改革和适当调整战略部署。目前，海外贸易是国家经济的重要组成部分，其主要渠道是海洋，

对海洋资源、空间的需求极高。我国虽然拥有着巨大的海洋发展潜力，是世界上少数拥有独立完善的海洋信息技术体系的国家，但是在核心技术、服务、应用等领域依旧面临许多问题。我国在国际上的优势主要依靠数据采集设备^[13]以及获取的数据量，在数据可视化方面，特别是高分辨率数据全域渲染研究较少。因此，本文基于 web 端提出了一种数据实时处理和渲染的方法，结合移动端控制，开发了一套图层基于大屏展示的可视化系统。系统采用分布式架构^[14]存储不同经纬度范围内的时变数据，在减少系统存储开销的同时也方便系统查询定点数据。系统利用大屏展示可视化效果，使用移动端控制可视化进程，简化用户操作，提供完善的交互服务，展示准确、详细的数据信息，为用户带来良好的使用体验。系统实现高分辨率数据实时、全域渲染，直观展现海洋信息的动态变化，充分挖掘海洋信息，帮助用户掌握海洋信息变化规律，对我国开发海洋资源，预测海洋天气，开展科研军事活动，占据未来发展主动权有着十分重要的意义。

1.2 国内外研究现状

海洋标量数据仅仅指代某一信息值的大小，不包含方向，数据来源有多种途径，包括温度、盐度、藻类密度等，是一种具有多源、海量、动态以及变化复杂等特点的海洋环境数据^[15]。海洋数据蕴含着十分复杂的时空过程以及多元要素的动态改变，为了直观展现隐含在数据中的变化规律，研究者探索出了多种可视化方法，研发了不同的动态可视化系统。

1992 年，联合国教科文组织政府间海洋学委员会在国际科学协会理事会、联合国环境规划署和世界气象组织的帮助下^[16]，为实现共享海洋探测信息，宣布开发一个全球海洋观测系统，这是目前世界上最大的海洋观测系统。

国外对于数据可视化系统的研究起步较早，现在已经研究出多种可视化方法，开发出多款优秀的可视化平台。Dunne^[17]等遵照 OGC 数据传输标准，将多波束声纳数据集成到 World Wind，实现了网络三维表达。层次细节处理 (Level of Detail, LOD) 最早由 Clark 于 1976 年提出^[18]，在不影响可视化效果的前提下，针对原始地形数据绘制不同层级下的模型，按照视线高低分别采用精细、粗糙的模型。LOD 动态调度算法公布之后，国内外研究者在此基础上进行改进，并将其运用在其他类型的数据可视化之上。最初的 LOD 算法基于 CPU，效率不高，随着 GPU 渲染技术的发展，结合 GPU 渲染的 LOD 动态调度算法^[19]开始在多种场景下崭露头角，如 Willem 提出的 Geomipmap Block LOD 算法，实现地形数据批量绘制，提高了渲染效率^[20]。

Google 于 2009 年推出的 Google Ocean^[21]，NASA 的 WorldWind^[22]，ESRI 的 ArcGIS Explorer^[23]以及开源的 osgEarth^[24]都是当下流行的虚拟地球平台。其中，Google Ocean 功能强大，利用三维地球球体实现海底地形三维可视化，但是缺少

对于海洋要素信息的表达。

国内研究虽然起步较晚,但发展速度快,逐渐缩小了与国外在研究水平上的差距。袁凌^[25]等基于 Cesium 和谷歌影像,搭建地形瓦片金字塔模型,在不同缩放层级下调用不同级别的地形瓦片,加载风机模型,实现风电场地形三维可视化;王想红^[26]等基于几何对象的可视化方法,结合 LOD 动态调度等方法实现海洋标量数据三维动态可视化,数据的分辨率是 $1^{\circ} \times 1^{\circ}$,在数据精细度上稍有欠缺;孙强^[27]等设计的基于 Cesium 的海洋标量数据可视化系统,根据使用者的视角高度来确定待渲染数据的分辨率,利用分块动态调度方法确定当前视野范围内的海洋数据块,进行可视化渲染,实现在网络环境下海洋标量数据的动态可视化。国家海洋信息中心于 2009 年发布 iOcean 中国数字海洋公众版^[28],实现中国海域多种海洋自然要素、海洋现象及其变化过程的数字化重现和立体展示。

GPU 渲染技术^[29]的不断完善以及渲染高效性,使其深受国内研究者的青睐。张信雪^[30]等分析读取 NetCDF 数据文件,结合 GPU 和 osgEarth 三维地球虚拟场景,基于 2.5 维面的颜色映射、数据转换和数据动态表达,实现基于文件存储的叶绿素 a 浓度的动态可视化;王琪^[31]围绕可视化展示技术和大屏可视化技术进行分析,设计实现基于 WebGL 的针对三维视图的 TDDVL 工具,实现三维数据可视化图层大屏展示,为不同行业用户提供可视化服务。

由于低分辨率数据在大屏上进行可视化展示时图层效果模糊,边缘锯齿明显,用户体验感较差,所以,针对高分辨率数据可视化,目前主流的方法是采用金字塔模式^[32]以及 LOD 动态调度算法:提前准备好不同层级下、不同数据源的图层切片,根据视域范围调用对应图层。其主要应用场景是对以往数据进行动态可视化分析,对特定范围的海洋信息进行查看。然而,随着海洋数据分辨率不断提高,可视化屏幕越来越大,实时应用场景不断增多,实现高分辨数据实时处理和全域渲染的需求以及可视化图层基于大屏展示的需求越来越迫切。

1.3 本文主要研究内容与论文组织结构

本文针对高分辨率海洋标量数据基于大屏展示的实时全域动态可视化,提供数据查询、图表展示等功能协助用户分析海洋信息变化规律,解决目前低分辨率数据可视化图层限制屏幕大小、数据展示过程中频繁与后台进行交互、时变过程中渲染了许多非必须渲染的数据从而降低渲染效率的问题;提出了一种基于 web 端的数据可视化方法,并基于 B/S 架构开发了一款移动端控制、大屏展示的可视化系统。

全文共分六章,包括引言、相关技术介绍、基于色差阈值的时变数据可视化方法、系统设计与实现、系统性能测试、总结与展望。

第一章:引言。本章主要介绍当前的时代背景与研究意义、数据可视化研究

现状，总结本文主要研究内容及组织。

第二章：相关技术介绍。本章阐述了 MQTT、WebSocket 两种通信协议、WebGL 渲染、Cesium 框架。本章对协议中的基本概念进行简单说明，介绍协议的主要特点和优势，分析协议的建立过程以及数据通信流程。叙述 WebGL 渲染管线，介绍 Cesium 框架和其对 WebGL 的封装。

第三章：基于色差阈值的时变数据可视化方法。本章主要介绍确定时变阈值使用的 CIEDE2000 色差计算公式的基本概念和计算公式；主要叙述为确定正确阈值所设计的测试程序以及开展的社会调研的主要内容，结合现有研究确定时变阈值。详细介绍本文为提高渲染效率所提出的数据可视化方法中的重要步骤和实现流程。

第四章：系统设计与实现。本章主要阐述系统的体系架构、系统层级、系统组成模块以及系统实现的功能。

第五章：系统性能测试。本章主要对高低分辨率数据可视化图层在大屏上展示的不同效果进行测试；对系统使用的通信模式、数据处理效率以及渲染效率进行测试，验证方法可行性。

第六章：总结与展望。对本文的研究内容进行归纳总结，对未来主要的钻研方向与内容进行展望。

本文逻辑路线图如图 1.1 所示。

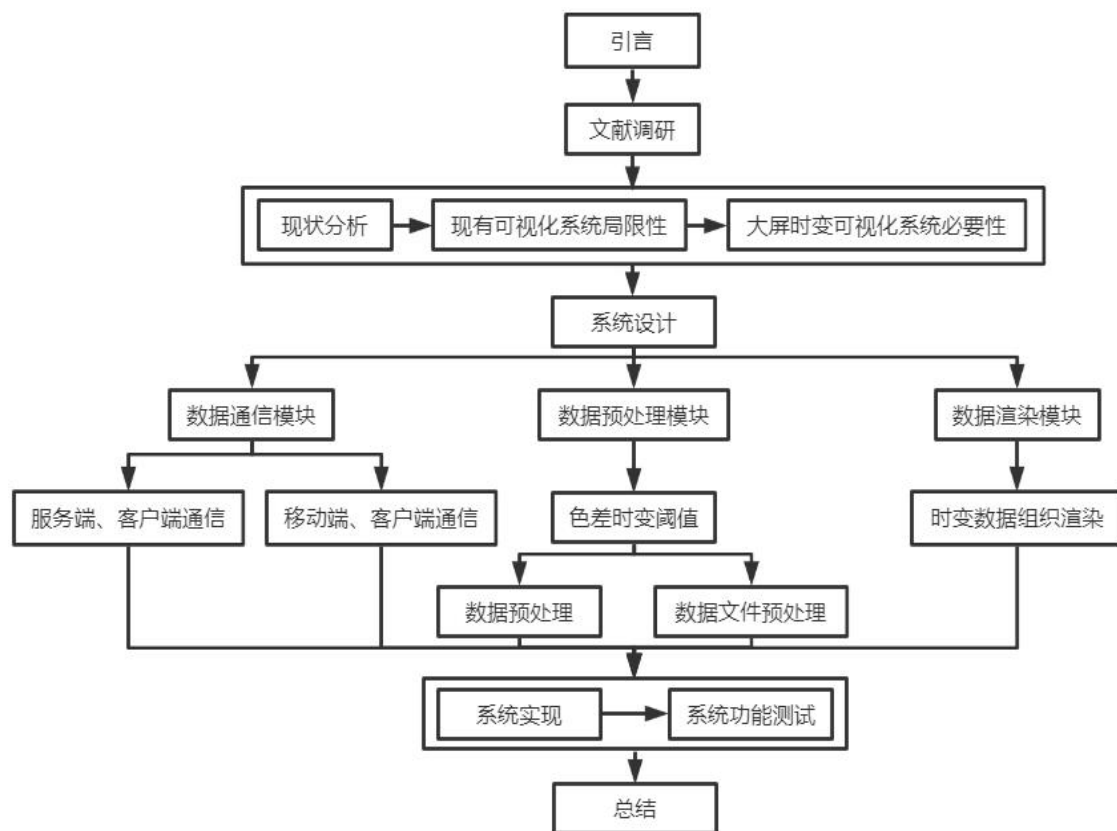


图 1.1 逻辑路线图

第二章 相关技术介绍

本章主要介绍系统涉及的关键技术点，包括数据通信技术、WebGL 渲染技术^[33]、Cesium 框架^[34]。数据通信技术包括移动端和客户端之间使用的 MQTT 通信^[35]以及服务端与客户端之间使用的 WebSocket^[36]通信。两种通信协议共同搭建起连通移动端、客户端和服务端的桥梁，实现三者之间的实时通信。本章主要介绍两种通信协议的基本概念、连接和通信原理以及技术优势，综合评价系统使用这两种协议的优势所在。分析对比 canvas, svg 以及 WebGL 三种渲染方式的优劣，介绍 WebGL 的基本概念和渲染流程，综合比对，最终选择使用 WebGL 渲染数据。介绍 Cesium 框架的基本概念，阐述 Cesium 加载数据模型的优势，详细说明 Render 模块的组成和作用。

2.1 MQTT 通信服务

2.1.1 MQTT 通信协议

MQTT 通信协议是由 IBM 和 Arlen Nipper 合作开发于 1999 年，起初是为了以电池消耗最少和带宽最少为代价，借助卫星连通石油管道^[37]。现如今，该协议的重点已由最初的嵌入式系统转换为物联网用例。该协议是一个基于发布/订阅模式的轻量级消息传输协议，服务于低带宽和网络环境较差的物联网应用，其主要特点是开放消息协议、发布/订阅模式、基于 TCP/IP 网络连接、报文结构紧凑以及可靠消息传输。较之基于请求/响应模式的 HTTP 请求，MQTT 由事件驱动，允许将消息推送给客户端，由此将多个客户端彼此分离，从而实现更高的可扩展性。具体内容如表 2.1 所示。

表 2.1 MQTT 协议主要特点

特点	描述
轻量级、开放	网络环境要求较低，减少带宽消耗
发布/订阅模式	事件驱动，提供了一对多消息分发和应用之间的解耦
可靠消息传输	三种 Qos 消息传递方式，保证数据传输
遗嘱消息	异常连接断开，发布遗嘱消息

2.1.2 MQTT 通信协议原理

MQTT 首屈一指的优点是：采用较少的代码和有限的带宽，为连通远程机器设备提供及时准确的消息服务，其原理流程图如图 2.1 所示。客户端、服务端之

间采用 MQTT 协议通信，其中包含三种定义：发布者、订阅者以及代理。此间，客户端充当发布者、订阅者两种身份，消息代理由服务端担任。

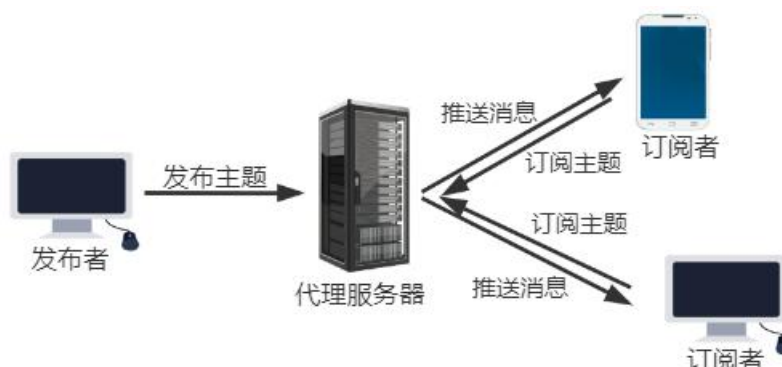


图 2.1 MQTT 协议原理流程

MQTT 协议基于 TCP/IP 协议，协议栈架构如图 2.2 所示，通信流程包括建立连接、客户端发布、客户端订阅以及服务端代理四个模块。建立连接模块主要为服务端和客户端之间建立通信连接，在订阅客户端通过心跳检测链路状态。客户端发布实现管理发布主题，与服务端多次握手，保证主题发送到服务端。客户端订阅实现管理订阅主题，与服务端多次握手，保证主题可以被客户端订阅。服务端代理作为通信桥梁，对连接、主题发布和订阅进行响应，实现消息传递。

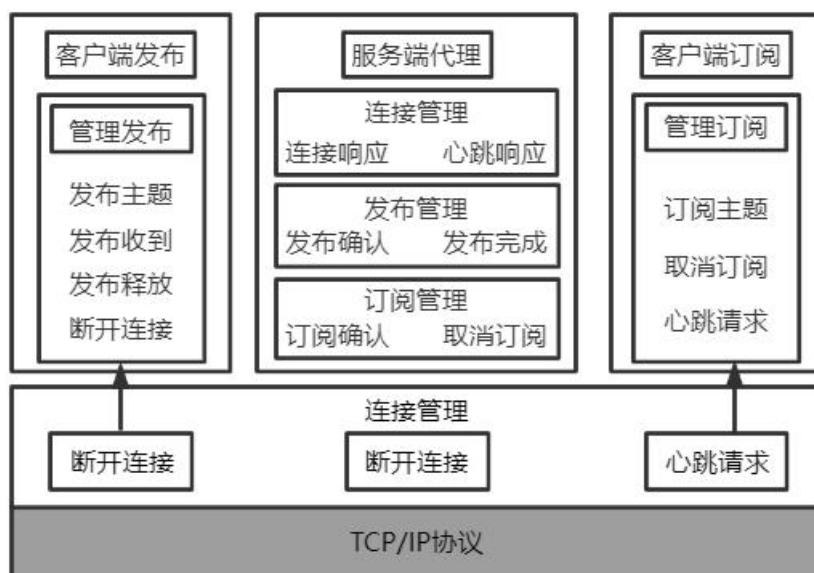


图 2.2 MQTT 协议栈架构

MQTT 通信实现的核心思想是发布/订阅模式^[38]。通信双方建立连接，由服务器发布一个主题，客户端订阅所需主题，从而以异步方式实现客户端与服务端之间的双向通信。彼此之间并不是直接连通开始交互数据，而是都与服务器建立

连接。服务器作为中间件，以发布/订阅事件为驱动，当一个客户端发布了某一主题的消息，服务器会传输给订阅此主题的其余客户端。通信双方无需关注对方状态，只需检测自身与服务器之间是否保持连接即可。主题发布之后，发布主题的客户端与服务器断开连接，无需同服务器保持长久连接，节省了带宽。订阅客户端则需要与服务器时刻保持连接，以秒为单位发送心跳检测报文，保障客户端与服务器断开连接后也能及时重新连接，从而保证消息传输。

为保证数据通信，该协议主要从两方面入手：其一是链路检测，通过定时收发心跳报文，当发生断链时通知双方知道此事件；其二是引入 QoS，保证由于网络故障发生丢包后，通知通信双方重新发送报文，保证消息正确传递。QoS 为消息发送端、消息接收端之间的协议，有三种不同层次，包括 QoS0：消息至多传递一次，QoS1：消息至少传输一次以及 QoS2：消息一次性准确传输。QoS0 模式下，消息一旦发出，就不再关心是否发送到对方，没有设置任何重发机制；QoS1 模式包含简单的重发机制，发送者发送消息之后等待接收者的 ACK，如果没收到 ACK 则重新发送消息。这种模式保证消息至少到达一次，但无法保证消息重复；QoS2 设计了消息重发和消息重复发现机制，保证消息发送给对方且只到达一次。不同层次的 QoS 传递流程图如图 2.3 所示。

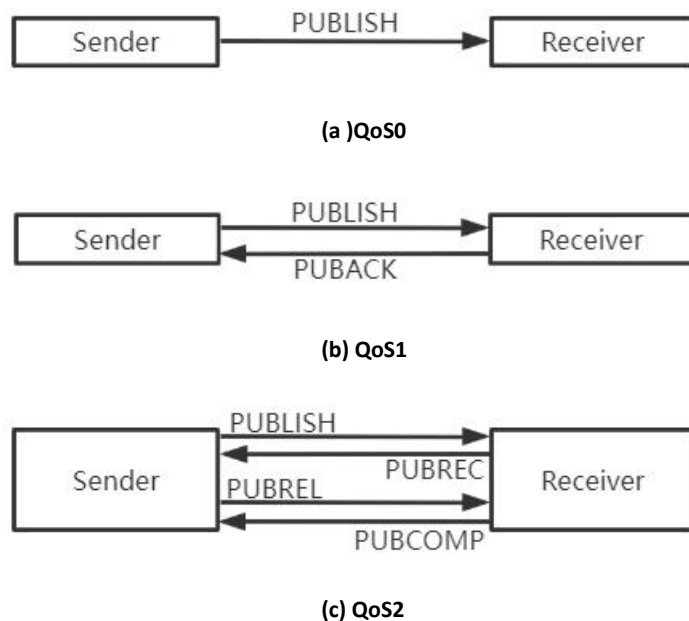


图 2.3 不同层次 QoS 传递流程

2.1.3 MQTT 报文结构

使用 MQTT 协议通信时，利用交换控制报文来实现数据通信。如图 2.4 所示，控制报文组成结构有固定报头、可变报头、有效载荷三部分。



图 2.4 MQTT 报文结构

固定报头是每一个 MQTT 控制报文最先被识别的部分,是任何一个控制报文不可或缺的一部分,需要 16Bit(位)占位数,即 2Byte(节)。

报文类型用于标识类型,位于第一个字节,表示为 4 位无符号值。每种报文类型以及功能描述如表 2.2 所示。第一个字节剩余的 4 位包含每个报文类型特定的标识,如果收到非法标识,消息接收方必须关闭网络连接。

表 2.2 报文类型

名字	值	报文流动方向	描述
CONNECT	1	Client ---> Server	客户端连接到服务器
CONNACK	2	Server ---> Client	连接确认
PUBLISH	3	Client <--> Server	发布消息
PUBACK	4	Client <--> Server	发布确认
PUBREC	5	Client <--> Server	消息已接收(QoS2 第一阶段)
PUBREL	6	Client <--> Server	消息释放(QoS2 第二阶段)
PUBCOMP	7	Client <--> Server	发布结束(QoS2 第三阶段)
SUBSCRIBE	8	Client ---> Server	客户端订阅请求
SUBACK	9	Server ---> Client	服务端订阅确认
UNSUBSCRIBE	10	Client ---> Server	客户端取消订阅
UNSUBACK	11	Server ---> Client	服务端取消订阅确认
PINGREQ	12	Client ---> Server	客户端发送心跳
PINGRESP	13	Server ---> Client	服务端回复心跳
DISCONNECT	14	Client ---> Server	客户端断开连接请求

剩余长度指明控制报文所剩其余字节数,包含可变报头与承载的真实数据,不包含编码其自身的字节个数。该字段表示的数据长度的编码方式不是固定的,编码方式的长短取决于实际数据长短。

可变报头位于固定报头与有效载荷之间。并不是所有报文都包含可变报头,

其内容根据报文类型发生相应变化，主要作用是作为包的标识，区分不同报文。客户端对一类报文多次发送时，传输过程中中断或需要多次重发，客户端就会根据每次发送情况来修改报文标识符。

有效载荷同样不存在于所有报文，其中的信息除了消息内容，还包括客户 ID、用户信息、服务质量要求、主题名等数据信息，并保证这些信息的正确性。其主要应用于 CONNECT、SUBACK、SUBSCRIBE 和 UNSUBSCRIBE 四种类型的报文。

2.2 WebSocket 通信服务

2.2.1 WebSocket 通信协议

WebSocket 协议诞生于 2008 年，于 2011 年成为国际标准^[39]。该协议诞生之前，为实现消息实时通讯，主要使用 HTTP 协议。HTTP 协议是一种无状态、无连接的单向通讯协议，采用请求/响应模式，由客户端向服务端发起通信请求，服务端响应之后向客户端发送数据。这就导致了一个弊端：服务端无法主动向客户端发送数据，在面对服务端数据变化频繁的情况下，客户端无法及时获取实时数据。为此，许多 Web 系统被迫选择频繁发送 Ajax 请求建立轮询或者长轮询的方式，这就浪费了大量资源。

WebSocket 是 HTML5 新增协议，归属于应用层，由客户端请求服务端实现双向连通，服务端成功响应请求，建立长时间连接。服务端可以主动发送数据给客户端，实现真正意义上的全双工通信。WebSocket 协议相比于轮询，其优势在于：允许数据双向传递，实时性更强；数据格式轻量，性能消耗小，效率更高。

2.2.2 WebSocket 通信协议建立

WebSocket 协议基于 TCP 协议^[40]实现。协议建立流程如图 2.5 所示，需要依赖 HTTP 协议实现初始握手过程，连接建立后数据通信不再依赖于 HTTP 协议。

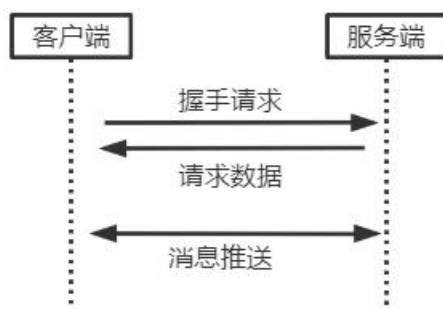


图 2.5 WebSocket 协议建立流程

服务端收到来自客户端的 HTTP 请求，双方“握手”建立通信。请求信息如图 2.6(a)所示，服务端收到请求之后，解析请求头字段，通过 `Connection:Upgrade` 和 `Upgrade:websocket` 字段升级 HTTP 协议为 WebSocket 协议，根据请求消息中的 `Sec-WebSocket-Key` 字段信息进行 base64 编码，生成一个新编码，写入响应标头信息 `Sec-WebSocket-Accept` 中发送给客户端，客户端匹配成功之后连接建立，响应标头信息如图(b)所示。自此，除非一方关闭连接，否则连接将一直存在。

▼ Request Headers [view source](#)

```

Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Cache-Control: no-cache
Connection: Upgrade
Host: 127.0.0.1:3001
Origin: http://localhost:3000
Pragma: no-cache
Sec-WebSocket-Extensions: permessage-deflate; client_max_window_bits
Sec-WebSocket-Key: 0Lm42gH+h4vEtQZpHNVu3Q==
Sec-WebSocket-Version: 13
Upgrade: websocket

```

(a) 请求标头

▼ Response Headers [view source](#)

```

Connection: Upgrade
Sec-WebSocket-Accept: HazsRhML5pMVHskCxQ3YywWoFXM=
Sec-WebSocket-Extensions: permessage-deflate
Upgrade: websocket

```

(b) 响应标头

图 2.6 请求、响应头信息

如图 2.7 所示，WebSocket 协议只支持 GET 方法，状态码返回 101，说明 HTTP 协议成功转换，WebSocket 协议建立完成。


```
Request URL: ws://127.0.0.1:3001/socket.io/?EIO=3&transport=websocket&sid=5gaYVgyuLl_vw-gJAAAA
Request Method: GET
Status Code: 101 Switching Protocols
```

图 2.7 协议升级请求

2.3 WebGL 渲染

前端渲染中，有很多方式可以用于绘制图像或动画，比如 canvas，svg 以及 WebGL。

Canvas，即画布标签，在 HTML5 阶段被提出，用于实现动态绘图。Canvas 标签元素必须指定宽高，提供各种用于绘图的 API，通过 JavaScript 编程调用 API 进行动态绘图。其优势在于绘制进程可以用 JavaScript 编程控制，具有较为灵活的逻辑处理能力，可定制性较强。相比于 svg 动画需要操作多个 HTML 元素来说，canvas 只在一个标签元素上操作，因此，在绘制复杂图形或动画时，canvas 性能较好。其劣势则是依赖像素，无法高效保真，画布增大会导致消耗增加。如图 2.8 所示，由 canvas 渲染出来的图称为位图，svg 渲染出来的被称为矢量图。



(a) 位图



(b) 矢量图

图 2.8 位图、矢量图比较

Svg 被称为可缩放矢量图形，是一种 XML 应用，可以用一种简洁，可移植形式表示图形信息。其功能丰富，不仅包含矩形、多边形等基本图形，同时还包括 fill，stroke 和 transform 等属性，实现各种动画效果，也支持通过 js 和 css 进行设置。Svg 的优势在于：本身作为一个矢量图，并不依赖像素，增大时并不会失真；由 DOM 驱动，由浏览器将事件绑定到不同的节点上，可以更好地实现动画控制。劣势在于：由于受到 DOM 操纵，通过 svg 实现复杂动画时，DOM 操作较为频繁，随着动画的复杂性增加，性能消耗也逐渐增大。Svg 和 canvas 之间的性能对比如图 2.9 所示，随着屏幕增大，canvas 渲染对性能消耗要高于 svg 渲染；随着渲染项目数量的增多，svg 渲染对性能消耗要高于 canvas 渲染。

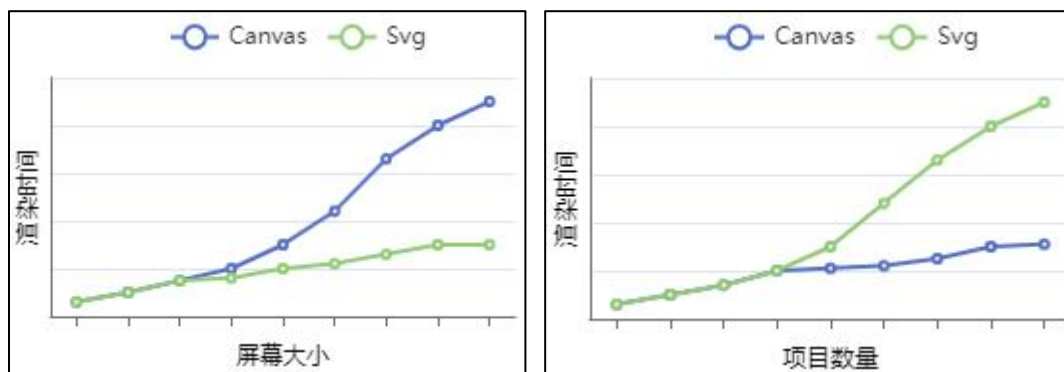


图 2.9 Canvas 和 Svg 性能对比

WebGL (Web Graphics Library), Web 图形库, 无需借助诸如 Flash, SilverLight 等浏览器插件, 帮助开发者渲染高性能的交互式 2D 或 3D 图形。WebGL 标准的问世使得开发者通过更加统一、标准的方式绘制 3D 图形。使用 WebGL 渲染 2D 图形是直接调用底层接口, 实现硬件加速, 拥有着比 canvas 和 svg 更加良好的性能。

GLSL (OpenGL Shading Language), 即 OpenGL 着色器语言, 是一款用于开发在 GPU 中执行着色器程序的语言, 用于取代 GPU 固定管线部分功能。开发者可以通过编程控制 GPU 渲染进程。主要功能是接收 CPU 传递过来的实现页面图形的渲染数据, 对这些数据进行流水线处理, 展示在屏幕上, 实现 3D 渲染。

图形渲染过程被称为渲染管线, 主要功能包括: 将物体 3D 坐标转换为屏幕 2D 坐标以及为屏幕每个像素点着色。渲染管线流程如图 2.10 所示, 包括: 顶点处理、图元组装和裁剪、光栅化和片元处理。



图 2.10 渲染管线

顶点处理允许操作顶点缓冲区内存储的顶点, 可在此阶段进行编程。转换顶点坐标, 将对象在原有的坐标系中的坐标, 转换为照相机所在坐标系中的坐标。

顶点处理完毕, 将这些顶点组成一个个形如点、线、三角形这类单元, 即图元, 该过程称为组装。而裁剪则是指对视口外的对象进行裁剪, 这个对象就是片元。

图元组装和裁剪完成之后, 进行光栅化处理。该过程就是将立体几何图像转为平面图像的过程, 决定窗口坐标中哪些整型栅格区被基本图元占用, 并在对应区域分配色值和深度值。主要目的是将图元数据转换为像素。但是, 光栅化处理后所得到的只是片元, 并不是像素。

片元是一个像素大小的基本单位，相比于像素，除了 RGBA 颜色值之外，还包含深度值、法线、纹理坐标等信息。为了得到像素，需要进行片元处理。该过程主要通过片元着色器计算片元的最终颜色、深度，筛选适合片元，去除法线、纹理坐标等，生成像素。

渲染管线处理流程如图 2.11 所示。

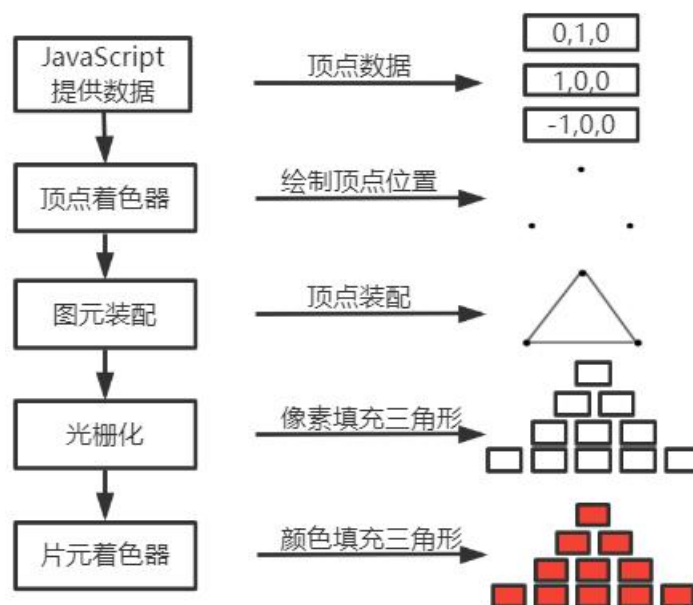


图 2.11 渲染管线处理流程

2.4 Cesium 框架

Cesium 是一个支持跨平台、跨浏览器，应用广泛的三维 GIS 框架，是一款基于 Apache2.0 许可的开源程序。使用 WebGL 进行硬件加速图形，同时无需插件辅助。

Cesium 功能强大，支持多种方式加载图层以及图层之间的快速切换，允许调用 OGC 空间数据服务规范下的 WMS、WMTS、TMS 以及 ArcGIS 图层服务。同时，Cesium 支持在三维球体上创建加载实体、模型以及各种数据，包括 GeoJSON、CZML、KML 数据。Cesium 对于数据量庞大的 3DTiles 模型数据的加载优化是其重要优势之一，通过 Cesium 内部的多线程机制运算，采用 WebGL 渲染提高模型加载速度，使得 3DTiles 成为了 Cesium 的御用格式。

Cesium 功能层级^[41]如图 2.12 所示，自上而下可分为 Web 应用层、动态场景层、场景层、渲染器层以及核心层。



图 2.12 Cesium 功能层级

核心层主要提供基础数学运算，比如投影、经纬度坐标转换以及矩阵运算，为图层加载、数据加载、实体模型加载等提供运算基础。

渲染器层对 WebGL 渲染进行封装，包括内置 GLSL 功能、着色器存储、顶点数组、渲染命令、纹理和渲染状态以及其他相关功能。图 2.13 所展示的是 Cesium 图形渲染器架构，主要组件包括：VertexArray、RenderState、ShaderProgram 和 Frambuffer。其中 VertexArray 类包含顶点数据和索引数据，两者存储在 Buffer 对象中；RenderState 包括发出 drawcall 所需的 WebGL 状态数据，如混合、深度和状态；ShaderProgram 表示经过编译、连接的 WebGLProgram。Uniform 可以和 Cesium 中的矩阵、笛卡尔变量、纹理等直接运作；Frambuffer 是帧缓存对象，是 drawcall 所需的素材。

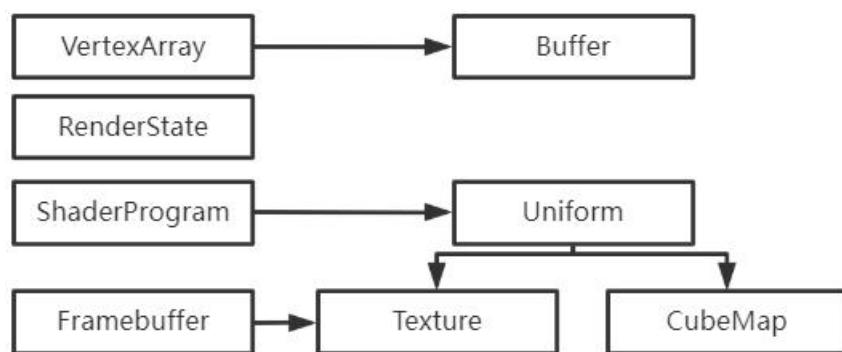


图 2.13 渲染器架构

场景层主要功能为加载多种数据原图层、模型以及构建照相机视角等场景。

动态场景层主要功能体现在数据动态展示方面，对 JSON、GeoJSON、CZML 等数据加载解析，以时间轴的方式，随着时间变化实现实时渲染和动态展示。

2.5 本章小结

本章主要介绍系统所使用到的 MQTT、WebSocket 两种通信协议、WebGL 渲

染技术、Cesium 框架。关于 MQTT 通信协议，简单介绍其基本概念，对该协议的主要特点进行简要概述；详细介绍该协议的通信原理和通信流程，介绍通信过程中的三种身份，分析说明 MQTT 协议的建立过程以及数据通信的核心思想，阐述为保证数据通信的正确性所使用的服务质量机制；详细说明 MQTT 协议报文格式三大组成部分。关于 WebSocket 协议，简单介绍其基本概念和优势，详细介绍该协议的建立过程。

关于 WebGL 渲染技术，主要介绍其渲染流程和渲染过程中的重要组成部分，同 canva, svg 进行对比分析，阐述本文使用 WebGL 渲染的原因。关于 Cesium，简单叙述了其基本概念和数据加载方面的优势，介绍了 Cesium 中对于 WebGL 渲染的封装模块。

第三章 基于色差阈值的时变数据可视化方法

本章主要叙述基于以往人眼色差识别的研究结果,结合调查问卷统计结果和 CIEDE2000 色差公式^[42]的计算结果,确定颜色变化阈值,即时变阈值。阐述时变数据动态可视化流程:结合时变阈值筛选变化数据,按照时变数据结构对变化数据进行组织预处理,更新渲染命令,从而减少时变渲染数据量,去除冗余数据,提高时变渲染效率。本章主要介绍 CIEDE2000 色差公式基本概念、确定时变阈值的方法和组织社会调查的方案、时变数据结构、预处理流程以及初始和时变渲染流程。

3.1 色差时变阈值

3.1.1 CIEDE2000 色差公式

人眼对于每种颜色变化的分辨能力存在差异,且识别每种颜色变化是有范围的。在颜色变化微弱的情况下,人眼无法观察到颜色变化,只有颜色变化达到某一阈值以上,人眼才可以观察到清晰的颜色变化。

CIEDE2000 色差公式是目前最为先进的工业色差公式,相比于 CIE94 或 CMC 都有了明显的改进^[43]。CIEDE2000 对比 CIE94,主要从以下几个方面做出了改进:

- (1) 修改标定中性区域 a^* 轴,从而提高预测中性色的能力。
- (2) 修改 CIE94 公式中的明度权重函数为近似 V 型函数。
- (3) 色相权重函数中融合色相角,体现色相容限随颜色色相而变化。
- (4) 融合 BFD 和 Leeds 色差公式中相似的椭圆选择项,修正蓝色区域的色差宽容椭圆不指向中心点的问题。

这些修正使得 CIEDE2000 色差公式在计算色差时更准确,也更加符合人眼对于颜色变化的识别能力,其完整公式如下:

$$\Delta E = \sqrt{\left(\frac{\Delta L'}{K_L S_L}\right)^2 + \left(\frac{\Delta C'}{K_C S_C}\right)^2 + \left(\frac{\Delta H'}{K_H S_H}\right)^2 + R_T \left(\frac{\Delta C'}{K_C S_C} \frac{\Delta H'}{K_H S_H}\right)} \quad \text{公式 3-(1)}$$

$$\left\{ \begin{array}{l} S_L = 1 + \frac{0.015(L' - 50)^2}{[20 + (L' - 50)^2]^{1/2}} \\ S_C = 1 + 0.045\bar{C}' \\ S_H = 1 + 0.015\bar{C}' T \\ T = 1 - 0.17 \cos\left(\bar{h}' - 30\right) + 0.24 \cos\left(2\bar{h}'\right) + 0.32 \cos\left(3\bar{h}' + 6^\circ\right) - 0.20 \cos\left(4\bar{h}' - 63^\circ\right) \\ R_T = -\sin(2\Delta\theta)R_C \\ \Delta\theta = 30 \exp\left\{-\left[\left(\bar{h}' - 275\right)/25\right]^2\right\} \\ R_C = 2\left(\frac{\bar{C}'^{1/7}}{\bar{C}'^{1/7} + 25^{1/7}}\right)^{1/2} \end{array} \right. \quad \text{公式 3-(2)}$$

其中 K_L 是调整明度系数, K_C 是饱和度系统, K_H 是色相的相对容量系数, 一般取值 1, 是实验条件校正的关键参数; $\Delta L'$ 为明度差, $\Delta C'$ 是彩度差, $\Delta H'$ 则为色调差; S_L 、 S_C 以及 S_H 是加权函数, 表示亮度补偿、色度补偿和色调补偿, 更正颜色空间的均匀特性; R_T 是色调差与彩度差的交叉项修正系数; R_C 为旋转函数。

3.1.2 时变阈值实验设计

设计颜色变化程序进行实验, 设计如图 3.1 所示。实验颜色包括红色、蓝色、紫色、棕色和黄色, 每种颜色对应的要素值区间再细分出三种深浅不一的颜色, 每组区间选取 0.01, 0.03, 0.05, 0.08, 0.1, 0.3, 0.5, 0.8, 1 九个不同的差值作为候选阈值。

每个颜色区间选定一种颜色作为标准色, 该区间内 9 个差值所对应的颜色作为对比色按序同标准色对比。实验参与者依次观察两个颜色, 记录下是否观察到颜色变化以及颜色变化是否明显。观察完毕之后, 填写调查问卷, 调查问卷部分内容如图 3.2 所示, 选择是否可以观察到颜色变化, 如果观察到较为明显的颜色变化则填写‘明显’。



图 3.1 实验设计

颜色变化阈值调查

*1. 蓝色第一区间差值0.01时是否可以观察到颜色变化?

- ☐ 可以观察到
- ☐ 没有观察到

*2. 蓝色第一区间差值0.03时是否可以观察到颜色变化?

- ☐ 可以观察到
- ☐ 没有观察到

*3. 蓝色第一区间差值0.05时是否可以观察到颜色变化?

- ☐ 可以观察到
- ☐ 没有观察到

图 3.2 调查问卷内容

为了保证社会调研的准确性，随机选择 50 人，进行多次问卷调查。每次调查相隔一天，照顾参与者情绪，去除长时间观察导致疲劳引起的误差，记录观察到颜色变化的人数。同时，记录 CIEDE2000 公式计算所得的每种差值下两个要素值对应的两种颜色的色差结果，结合调查问卷统计结果，同之前的人眼色差识别研究结论作比较，检验本文所得结果是否正确，确定最终的时变阈值。

3.1.3 时变阈值选择

每种颜色区间可以观察到颜色变化的人数和不同差值下利用 CIEDE2000 公式计算结果如表 3.1 所示(说明：上为观察到颜色变化人数，下为 CIEDE2000 计算得到的色差值)。

表 3.1 不同差值观察到颜色变化人数及对应色差值

差值	要素值颜色														
	蓝色 1	蓝色 2	蓝色 3	紫色 1	紫色 2	紫色 3	黄色 1	黄色 2	黄色 3	棕色 1	棕色 2	棕色 3	红色 1	红色 2	红色 3
0.01	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0.017	0.015	0.021	0.041	0.115	0.153	0.155	0.068	0.062	0.078	0.093	0.070	0.022	0.021	0.022
0.03	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0.051	0.046	0.064	0.122	0.346	0.460	0.358	0.201	0.186	0.234	0.282	0.257	0.065	0.064	0.064
0.05	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0.086	0.077	0.108	0.204	0.577	0.767	0.590	0.331	0.312	0.391	0.469	0.428	0.109	0.106	0.108
0.08	0	0	0	0	0	21	0	0	0	0	0	0	0	0	0
	0.138	0.123	0.173	0.329	0.928	1.227	0.942	0.527	0.499	0.627	0.750	0.685	0.174	0.171	0.173
0.1	0	0	0	0	38	50	0	0	0	0	0	0	0	0	0
	0.172	0.154	0.217	0.412	1.164	1.534	1.174	0.656	0.604	0.701	0.937	0.858	0.217	0.214	0.217
0.3	0	0	5	50	50	50	50	50	50	50	50	50	4	0	2
	0.516	0.464	0.669	1.276	3.560	4.611	3.447	1.945	1.836	2.125	2.797	2.547	0.651	0.642	0.652
0.5	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
	0.859	0.778	1.150	2.199	6.163	7.667	5.612	3.196	3.102	3.583	4.623	4.196	1.083	1.071	1.085
0.8	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
	1.373	1.259	1.928	3.698	10.18	12.14	8.686	5.015	5.193	5.825	7.280	6.585	1.728	1.716	1.734
1.0	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
	1.532	1.588	2.490	4.779	12.91	15.02	10.622	6.192	6.568	7.362	8.984	8.113	2.157	2.147	2.166

结果显示，参与者对于红色和蓝色两种颜色的颜色变化分辨能力较强，对于其他三种颜色的颜色变化辨别能力较差，对于黄色的颜色变化分辨能力最差。其中，紫色和棕色在要素值差值为 0.1 时观察到颜色变化。表 3.2 展示了取 NBS 色差单位^[44]时，不同色差值范围下对颜色变化的感知能力；罗立鸿^[45]基于亮度改变后，不同颜色利用 CLELAB、CIEDE2000 两种公式计算色差，可以明显观察到颜色变化的色差阈值如图 3.3 所示。CIEDE2000 计算结果显示色差大于 1.0 时可以观察到明显的颜色变化。本文所得到的实验结果与之相符，从而证实实验调研的正确性，因此，为了覆盖更多数据，最大程度保证可视化效果的情况下选择 0.1 作为时变阈值。

表 3.2 NBS 单位色差值范围感觉色差程度

NBS 单位色差值	色差感觉程度
0.0-0.50	(微小色差)感觉极微
0.5-1.50	(小色差)感觉轻微
1.5-3.0	(较小色差)感觉明显
3.0-6.0	(较大色差)感觉很明显
6 以上	(大色差)感觉强烈



图 3.3 基于 CIELAB、CIEDE2000 下不同颜色色差值

3.2 时变数据可视化

3.2.1 时变数据结构

系统针对分辨率为 0.05° 的高分辨率海洋标量数据，按图 3.4 所示的划分规则对数据进行切分。首先，系统按纬度范围 45° ，全经度范围将全域数据分成 4 块，用文件存储，每份数据文件存储在一台数据服务器中，用 i 表示，记作初次划分序号。数据服务器按纬度范围 11.25° ，经度范围 11.25° 对数据块进行切分，经度自左向右，纬度自下向上，分块序号用 j 表示，记作数据块序号。每一个数据块包含 225×225 个数据。

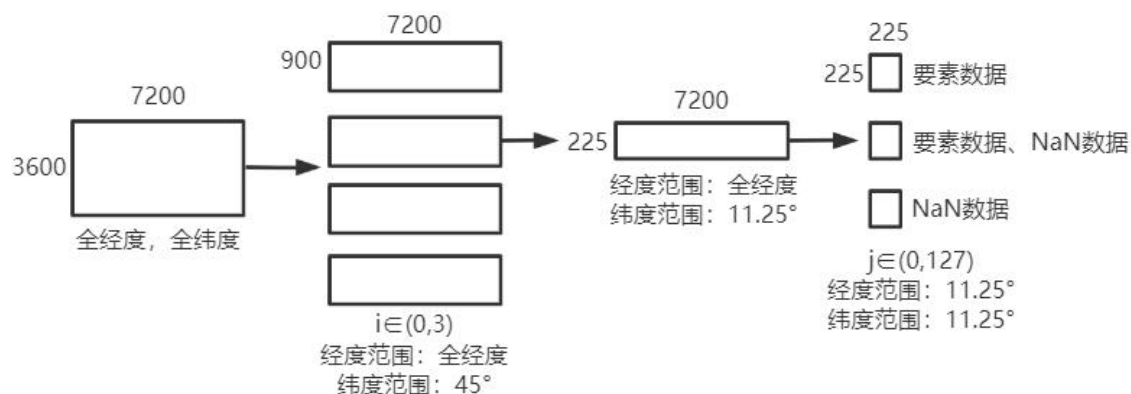


图 3.4 数据划分规则

按照数据划分规则设计时变数据结构，如图 3.5 所示。



图 3.5 时变数据结构

时变数据结构包含 4 部分：初次划分序号 i ，数据块序号 j ，要素值在数据块中的序号 k 以及要素值 $value$ ，其中， $k = 225 \times m + n$ （ m ， n 分别代表变化数据在数据块中的行下标和列下标）。

初次划分序号 i 对应文件存储所在数据服务器的标号。每台数据服务器独立并发处理数据文件，结合时变阈值筛选变化数据，计算变化数据在数据块中的序号 k ，记录变化数据值 $value$ ，组织为时变数据。服务器按照数据划分规则对时变数据进行最后的整理打包，相同范围内的时变数据在同一个时变数据块内，记录下时变数据块序号 j ，顺序发送给客户端。

经过筛选组织的时变数据，每次时变中去除了经纬度冗余数据，提高了数据的传输速度；能够准确定位需要更新的要素值，提高时变渲染的速度。

3.2.2 数据预处理

数据预处理有时变数据预处理和数据文件更新预处理两部分。

时变数据预处理是根据时变阈值筛选变化数据，组织成时变数据，并打包成时变数据块发送给客户端。服务器读取接收到的最新数据文件，根据数据划分规则划分多个数据块，记作比较数据，当前正在渲染的数据块记作被比较数据。服务器遍历对比两者中的数据，筛选变化数据。计算相同位置的两个数据之间的差值。若差值大于等于时变阈值，则比较数据中对应的数据值作为时变数据进行组织，与同范围内的时变数据打包成时变数据块，写入时变数据文件。

数据文件更新预处理是时变数据可视化方法中十分重要的一个环节，其确保了筛选变化数据的正确性。假设先前一次变化数据筛选的过程中，文件一和文件二中相同位置的要素值差值小于时变阈值，则要素值不进行更新渲染；当文件二和文件三对比筛选变化数据时，同样的位置，两个文件的要素值差值仍然小于时变阈值，同样不对其进行更新渲染。但是，文件一和文件三对比时，相同位置的两个要素值差值大于时变阈值，需要更新渲染，此时，变化数据的筛选就出现了错误。而数据文件更新预处理正是为了解决这一错误。

变化数据筛选完成，将先前的数据文件称为待比较数据，最新获取到的数据文件称为比较数据。循环遍历两者的数据值，进行差值计算。差值小于时变阈值，不做任何改变；差值大于等于时变阈值，则将待比较数据中对应位置的数据值替换为比较数据中相同位置的数据值，数据文件更新完成。更新后的数据文件作为待比较数据参与下一次数据文件更新，文件中的数据作为被比较数据参与筛选变化数据。预处理过程如图 3.6 所示。

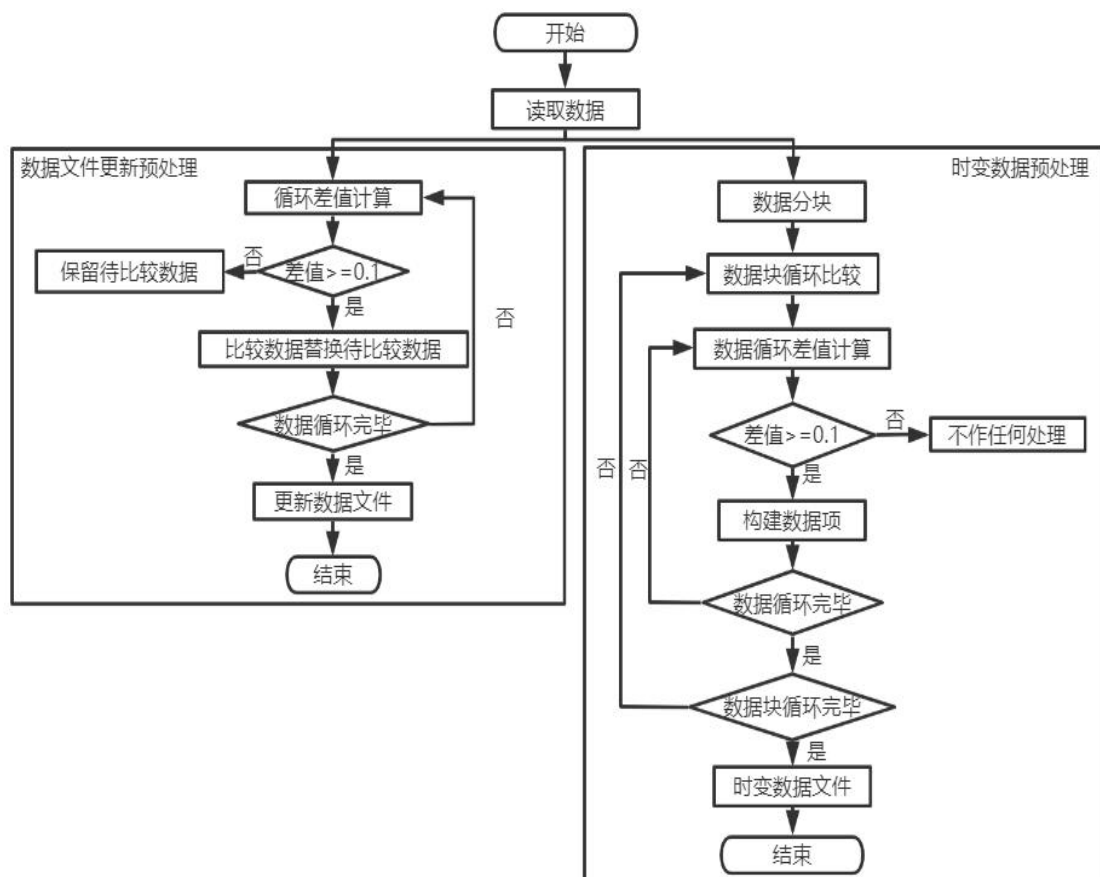


图 3.6 数据预处理

3.2.3 数据可视化

Cesium 框架中的 Render 模块对 WebGL 渲染进行封装，自渲染开始，时刻不停地请求新的渲染帧，实现动态渲染效果。任何几何对象所反映的都是空间中 N 个顶点之间的关系，利用 WebGL 渲染几何对象之前，需要将几何对象转换为 WebGL 可识别的格式：构建对象的顶点数组和索引数组。顶点数组包括点位置坐标，颜色，纹理等；索引数组则是几何对象顶点的索引信息，即顶点的先后顺序。

本文研究的系统利用 WebGL 调用 GPU 渲染空间几何对象，每一个几何对象由四个顶点组成，每个顶点包含经纬度、高度、海洋要素信息值对应的颜色信息。每一个几何对象代表不同的位置，利用不同颜色表示该点的要素信息值，所有独立的几何对象拼接组成全域海洋数据整体可视化效果，并利用色差时变渲染，展示动态可视化效果。

数据可视化分成两个阶段：数据初始渲染和数据时变渲染。每个阶段的渲染流程如图 3.7 所示。

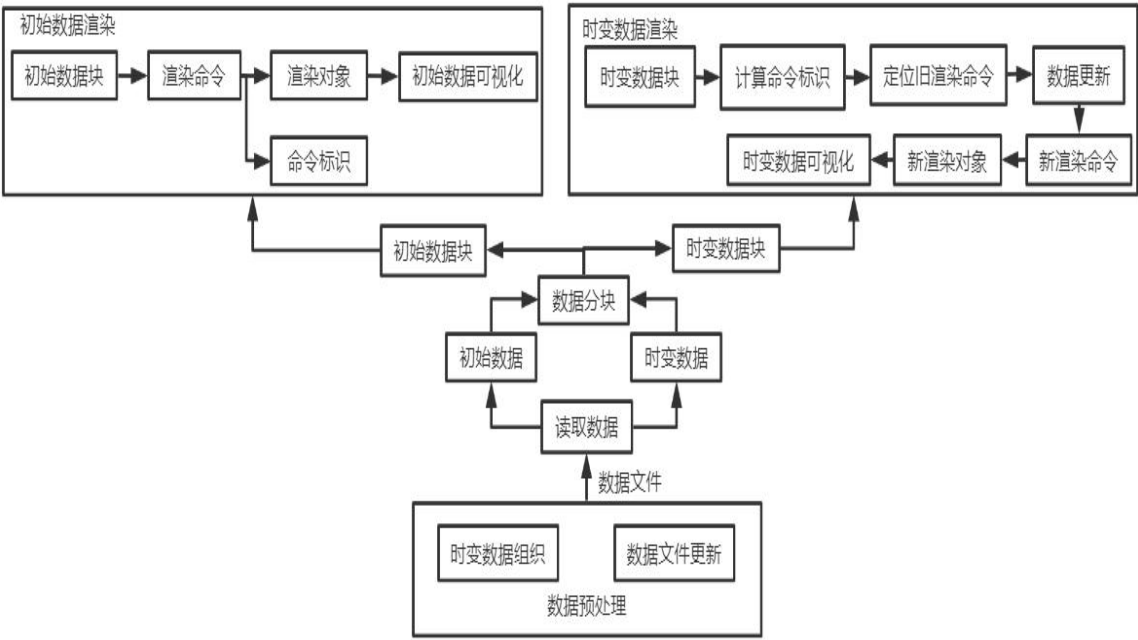


图 3.7 数据可视化渲染流程

数据初始渲染由服务器经过数据预处理，将打包好的初始数据块按序发送，交由前端渲染。为了降低数据传输过程中的带宽损失，提高数据的传输速度，一方面，系统使用 WebSocket 通信传输数据，减少通信消耗的同时提高系统实时性；另一方面，根据几何对象 4 个顶点之间的规律，初始数据结构如图 3.8 所示，Lon 代表几何对象右上角顶点经度，Lat 代表顶点纬度，value 表示该点的要素值。服务器只负责处理每个渲染对象中一个顶点的位置属性值，由客户端分析计算其余三个顶点的位置属性值。

Lon	Lat	value
-----	-----	-------

图 3.8 初始数据结构

客户端遍历初始数据，分析转换 4 个顶点的位置坐标，将要素值转换为 RGBA 数据，生成一条渲染命令。具体流程如图 3.9 所示。每一条渲染命令包含其所对应的经纬度范围内所有位置点的渲染对象，并有唯一的命令标识与之对应。命令标识为之后的时变渲染准确定位渲染命令起到至关重要的作用。每一个初始数据块都对应一条渲染命令，Cesium 的 Render 模块将每一条渲染命令添加到命令列表 commandList 中，每请求一次渲染帧，则对命令列表中的命令循环渲染一次，生成可视化图层，添加到三维球模型上。

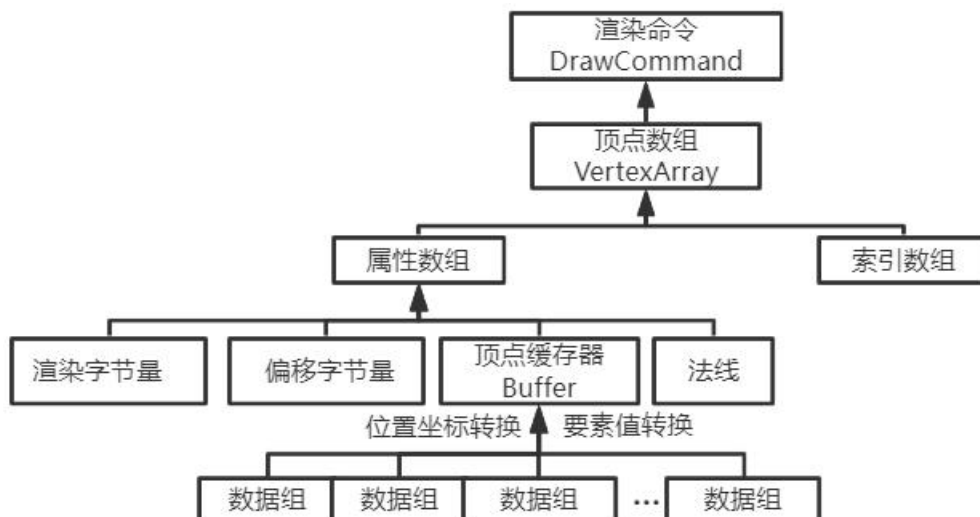


图 3.9 生成渲染命令

数据时变渲染结合时变阈值筛选变化数据减少了更新渲染数据量，使用时变数据结构组织时变数据去除了经纬度冗余数据，计算命令标识准确定位需要更新的渲染命令，实现数据按需更新，提高了数据传输速度和渲染效率。

本文研究的系统采用分布式架构，不同的服务器处理对应经纬度范围内的初始数据和时变数据，由此，渲染命令并非是有顺序的。客户端接收到服务端传递而来的时变数据包，循环遍历包中时变数据，利用 i, j 计算得到命令标识，计算公式为： $i \times 128 + j$ ，从而准确定位需要更新的渲染命令。 k 值用于精准定位待更新的要素值在命令中的位置下标。要素信息 $value$ 值转换为 RGBA 值完成数据更新，待更新的数据下标计算公式为： $28 \times k + 7 \times a + c$ ，其中， a 表示 4 个顶点的要素值依次更新； c 表示待更新的 R, G, B 值按序更新，A 值固定为 1 不需要更新。时变数据遍历完成，生成一条新的渲染命令，取代旧的渲染命令，渲染更新完成。

3.3 本章小结

本章主要介绍为确定时变阈值所进行的实验和社会调研，介绍 CIEDE2000 色差公式、实验方法和社会调研内容。对 CIEDE2000 色差公式的基本概念进行简单介绍，阐述与之前的色差公式相比其优化的内容和基本公式。主要介绍为确定人眼对于不同颜色的变化阈值所开展的实验方案以及社会调研内容，将其结果用表格记录下来，与之前被大众所认可的实验结果进行对比，结果表明本文得到的结论符合之前的研究结果，在最大程度保证可视化效果的前提下确定时变阈值。

阐述了本文所提出的数据可视化方法的流程和关键步骤。主要介绍数据划分规则、时变数据结构、数据预处理和数据渲染过程。根据系统使用的数据分辨率

和系统架构设计了数据划分规则，根据数据划分规则定义时变数据结构；预处理的数据包括时变数据和数据文件，主要介绍两种预处理流程和功能；数据渲染主要介绍初始渲染和时变渲染两种渲染方式的流程，分析了经纬度数据和要素数据转换以及时变数据如何准确定位更新位置，完成渲染命令按需更新。

第四章 系统设计与实现

本章主要介绍系统体系架构、系统模块设计以及系统所实现的各种功能。系统体系架构主要说明实现系统所用的框架和框架内各部分之间实现通信的方式。同时,描述系统的组成层级,说明每个层级的作用和功能。系统模块设计主要描述各模块实现方式、功能以及模块间的数据通信。介绍系统实现的各种功能,主要阐述系统功能的内部逻辑和具体实现方法,说明系统提供的交互服务的具体操作流程。

4.1 系统体系架构

4.1.1 系统框架

本文系统采用如图 4.1 所示的分布式框架,包括 1 台主服务器、1 台代理服务器以及 4 台数据服务器,1 个客户端,移动端可以是手机也可以是 pad。服务端使用 Node.JS^[46]开发语言,利用 JavaScript 开发客户端,使用 Vue.JS^[47]开发移动端。

主服务器直接与客户端相连,提供初始图层数据,提供 Cesium 基础图层服务和数据分析服务,实现基础图层的切换;加载三维地球模型,为渲染图层提供模型支持。

代理服务器作为移动端和客户端之间沟通的桥梁,用于响应连接、订阅和发布主题;传递实时消息,确保消息的准确传达。

数据服务器使用 WebSocket 协议与客户端通信,该类服务器只进行数据交互,因此称之为数据服务器。每台服务器独立提供接口,响应客户端请求,独立从数据收集设备接收不同经纬度范围的实时数据,经过时变数据预处理和数据文件更新预处理,打包时变数据块传输给客户端。服务器利用文件分散存储不同经纬度范围的数据,减少存储空间的消耗,提高数据的处理和查询速度。

客户端用于渲染实时数据,展示数据可视化效果,可连接大屏显示器,将可视化图层投影到大屏上。客户端使用 WebSocket 协议与数据服务器相连,请求时变数据、发送查询请求、获取响应数据;使用 MQTT 协议与移动端相连,及时响应移动端操作,同步展示移动端交互动作,方便用户操作。

移动端主要用于简化操作。使用者可以在手机端或 pad 端对三维地球模型进行控制,主要功能包括:控制渲染进程、查询定点要素信息、自定义范围划定等。

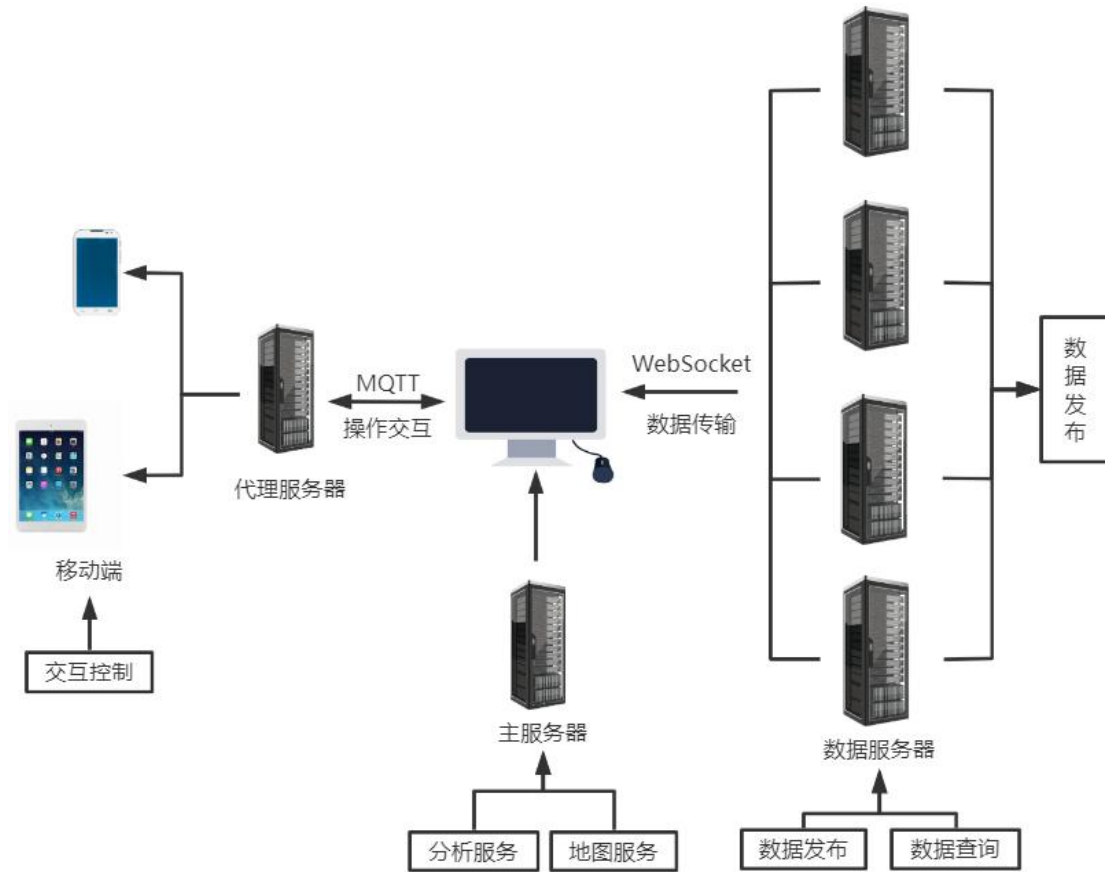


图 4.1 系统分布式框架

4.1.2 系统层级

系统层级如图 4.2 所示，自下而上包括：数据层、服务层、逻辑控制层、可视化层以及应用层。

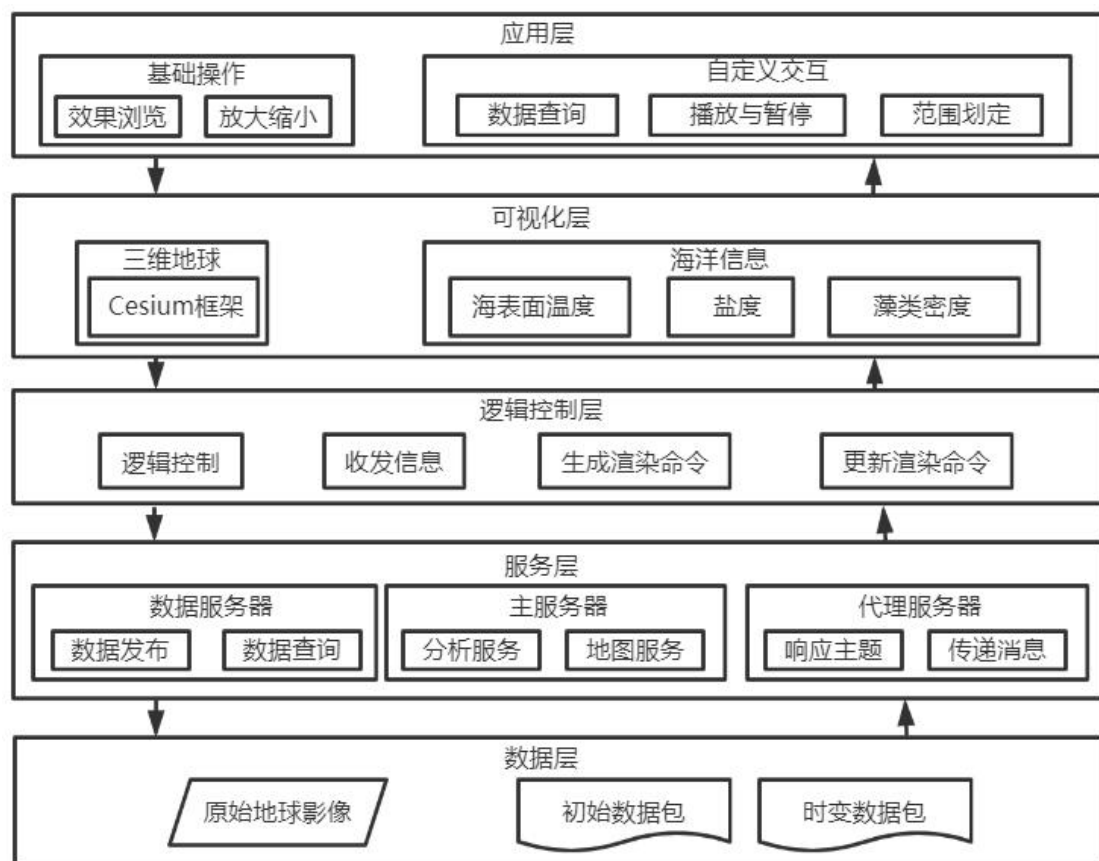


图 4.2 系统层级

数据层在典型的 B/S 架构中扮演抽象化数据为可编程对象的角色，作为数据提供者，主要提供基础的地球影像数据以及预处理好的海洋标量数据包，包括初始数据包和时变数据包。原始数据使用 NetCDF^[48] 文件进行存储，借助第三方工具转为 txt 或者 csv 文件存储，经过服务器读取、预处理之后转换为 JSON 格式用于数据传输。数据文件更新后存储在服务器中。

服务层作为分析、处理数据的核心层次，主要用于处理发布数据，提供数据传输、分析查询、连接以及主题发布订阅的接口，响应客户端请求。由于系统采用全双工通信模式进行数据交互，允许服务器主动向客户端推送数据，增强了数据实时性。服务器并发处理不同经纬度范围的数据，使用文件存储数据，保证处理数据的高效性，节省存储空间。客户端接收数据时，利用初次划分序号从指定服务器获取数据，提高数据的查询速度。

逻辑控制层作为服务层和可视化层的中间层，主要负责逻辑控制，包括指令的收发和判断，数据的格式转换、分析计算和动态发布，为数据可视化渲染提供强大的逻辑支持，保证数据传输和交互行为的正确性。MQTT 协议使用订阅/发布模式，引入 QoS 服务质量保证消息正确无误地传达到消息接收方；WebSocket 协议使用事件驱动、双向通信保证数据的正确性和实时性。客户端渲染过程中，

Cesium 内部对于 WebGL 渲染功能的封装保证了每一个渲染对象在正确的位置上完成快速渲染。

可视化层负责渲染各种类型的数据，加载可视化图层，展示数据动态变化，帮助用户准确地掌握变化规律。可视化图层大屏展示使用户把握更加精准的海洋信息，获得更佳的使用效果。

应用层直接面向用户，担任用户和系统的沟通桥梁，需要为用户提供良好的可视化效果和交互体验。除了提供基本的可视化效果浏览操作之外，该层也允许用户使用移动端控制渲染进程、进行数据查询以及范围划定。

4.2 系统模块设计

4.2.1 数据通信模块设计

系统数据通信模块由移动端和客户端之间的通信、服务端和客户端之间的通信两部分组成，其将三者连接成统一的整体，共同完成系统各个部分之间的数据交互功能。在面对需要频繁进行数据交互的场景，该模块需要提供可靠的数据通信管道，保证数据传输的正确性、实时性；在面对网络质量不佳的情况下也需要提供较为良好的数据交互服务。数据通信模块各部分面对的主要业务场景、亟需解决的问题不同，因此，需要采取不同的通信协议。

移动端与客户端需求分析图如图 4.3 所示，由于移动端和客户端之间需要同步操作，保持操作的一致性，所以两者之间需要进行频繁的数据交流。移动端的各种操作指令需要及时传递给客户端，由客户端响应指令，并完成同步操作。这就要求两者之间的通信方式要提供可以支持长连接通信的通信管道，保证数据交互的实时性，同时，需要保证指令传输的正确性，保证客户端正确响应来自移动端的不同指令。系统使用的 MQTT 协议提供可靠的、持久性的双向通信，即使面对网络不稳定的环境依然有较好的数据通信功能。同时，其自身采取的订阅/发布模式以及 QoS 服务质量进一步保证了数据传输的可靠性和正确性。

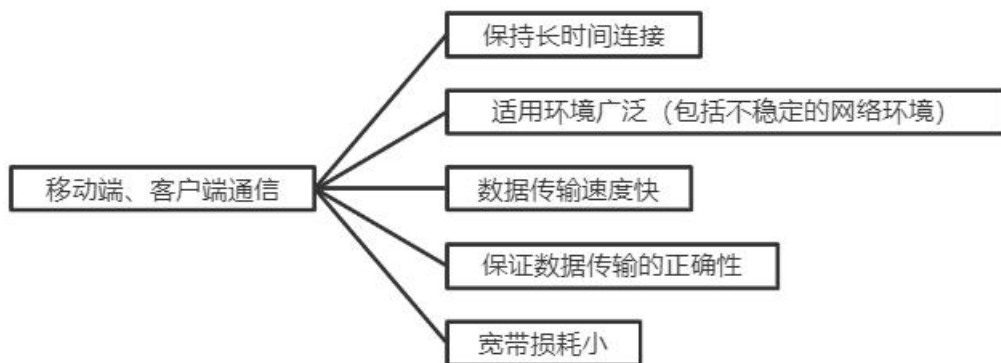


图 4.3 移动端与客户端通信模块需求分析

模块设计如图 4.4 所示，使用 EMQ X 提供的公共消息服务器作为移动端和客户端之间代理服务器，负责处理连接请求、响应处理主题的发布和订阅以及消息传递。移动端作为主题的发布者，不同操作对应不同主题，消息内容根据主题发送不同的属性和属性值，例如：改变视野范围，发送经纬度和高度；改变要素值类型，发送要素值种类名称。由于 EMQ X 提供的代理服务器所支持的消息内容只有数据流和字符串两种格式，所以，移动端在发布消息时先将内容对象转换为 JSON 字符串，客户端接收后再解析为 JSON 格式。客户端作为主题的接收者，根据主题接收对应的消息内容，转换数据格式，获取属性特征值，同步响应移动端操作。

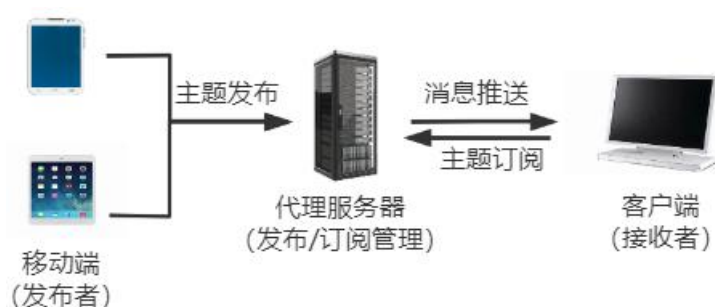


图 4.4 移动端与客户端通信模块设计

主要通信流程如下：

1. 建立连接。移动端和客户端分别使用 WebSocket 同服务器实现连接；设置心跳检测，检查连接状态。
2. 移动端发布主题。用户在移动端操作，触发不同的主题事件，携带对应的消息内容发布到代理服务器。代理服务器确认收到消息之后推送给订阅了该主题的客户端。

关键代码如下：

```

Client.publish(ThemeName,Message,{Qos},{err})=>{
  if(err){
    客户端连接断开触发
  } else {
    主题发布确认，消息传输成功
  }
}

```

3. 客户端接收消息。客户端在此之前需要订阅主题，并发送给代理服务器确认主题已订阅。客户端根据主题接收对应的消息内容，触发对应的事件函数做出响应。

订阅主题确认关键代码如下：

```
Client.subscribe(ThemeName,Message,{Qos},{err,granted})=>{
  if(err){
    客户端连接断开触发
  } else {
    订阅确认，返回主题名，服务质量
  }
})
```

接收消息关键代码如下：

```
Client.on('message',(topic,message,packet)=>{
  接受主题消息，返回主题名，消息内容，数据包
})
```

移动端改变视域范围时客户端接收到的消息内容以及数据包格式内容如图 4.5，图 4.6 所示。

```
Object { lon: 2.5675064088598893, lat: -0.16492941667671515, height: 75000183.93439001 }
  height: 75000183.93439001
  lat: -0.16492941667671515
  lon: 2.5675064088598893
```

图 4.5 消息内容

```
Object { cmd: "publish", retain: false, qos: 1, dup: false, length: 97, topic: "changePosition", payload: Uint8Array(79), messageId: 1 }
  cmd: "publish"
  dup: false
  length: 97
  messageId: 1
  payload: Uint8Array(79) [ 123, 34, 108, ... ]
  qos: 1
  retain: false
  topic: "changePosition"
  <prototype>: Object { ... }
```

图 4.6 数据包

服务端与客户端通信需求分析如图 4.7 所示。服务端和客户端面临的数据交互频率较为稳定，服务器按固定时间间隔接收来自数据收集器发送的数据文件，经过数据预处理之后，主动将处理好的数据包发送给客户端。因此，数据信息流

通图如图 4.8 所示。服务端与客户端之间的通信要求服务器不仅能响应来自客户端的请求，同时，服务器还能主动向客户端推送数据，实现数据的全双工通信，保证数据的实时性。此外，节约宽带开销，提高数据的传输速度也在通信方式选择的考量之内。

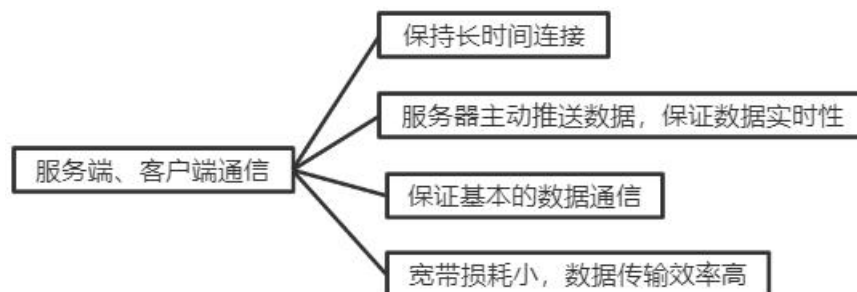


图 4.7 服务端、客户端通信需求分析

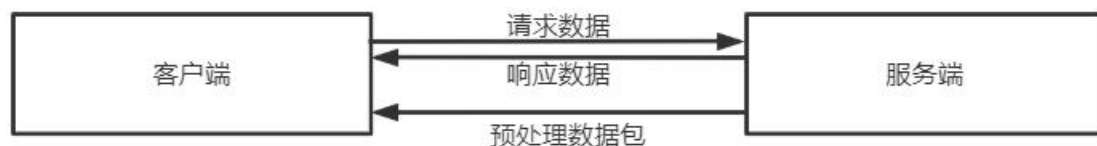


图 4.8 数据信息流通

不同通信方式之间的对比见表 4.1。经过比较，WebSocket 通信方式性能最佳，可以保证数据实时传输。因此，系统使用 WebSocket 通信协议连接服务端与客户端。

表 4.1 几种通信方式比较

	短轮询	长轮询	WEBSOCKET
浏览器支持	几乎所有现代浏览器	几乎所有现代浏览器	IE 10+, Firefox 4+, Chrome 4+
服务器负载	CPU 资源占用较少，内存资源与宽带资源占用较多	CPU 资源与宽带资源占用较少，内存资源占用较多	性能最佳，CPU 和内存资源以客户端事件数衡量，无需循环等待
客户端负载	占用较多内存资源与请求数	占用较多内存资源与请求数	以客户端事件数衡量
延迟	非实时，请求间隔决定延迟时间	非实时，请求间隔决定延迟时间	实时
实现复杂度	简单	需要服务器配合，客户端实现简单	需要 Socket 程序实现和额外端口，客户端实现简单

系统使用 Node.JS 创建服务器，引入 Socket.io^[49]实现服务端与客户端之间的 WebSocket 通信服务，由两部分组成，一部分挂载到 Node.JS 创建的 HTTP 服务器上，另一部分加载到客户端。具体创建、连接流程如下：

(1) Node.JS 创建服务器，负责读取文件、数据预处理以及响应各种客户端请求。

(2) 导入 Socket.io 数据包，挂载到服务器上，配置基础信息，完成 WebSocket 通信在服务端上的配置。

(3) 客户端加载 socket.io.js 文件，引入 socket 方法。

(4) 客户端发送 HTTP 请求，请求与服务端进行“初次握手”。服务端收到请求后，解析请求头，实现协议升级，并向客户端发送响应信息。

(5) 客户端匹配响应信息正确，WebSocket 通信连接建立。

连接通信流程如图 4.9 所示。

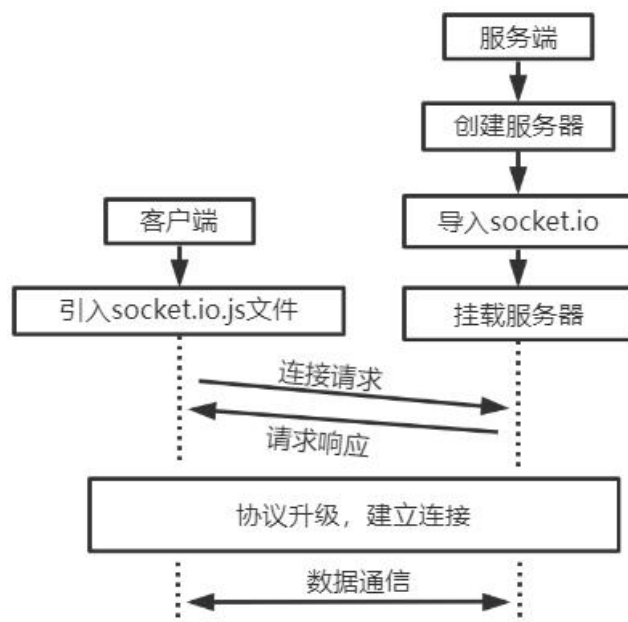


图 4.9 WebSocket 协议连接通信流程

WebSocket 协议利用事件监听机制^[50]建立连接、数据通信以及关闭连接。首先，服务端监听 connect 事件接收来自客户端的连接请求，建立 WebSocket 通信。连接建立之后，开始数据传输，包括两种方式：一种方式是通过 send 方法直接发送数据对象；另一种是通过 emit 方法，该方法需要两个参数，事件名称和传输的数据对象。前者由数据接收方监听 message 事件，利用回调函数接收消息发送方传递而来的数据对象；后者则是监听与声明事件名称相同的事件，通过回调函数接收消息。两者并没有本质的差别，不过，在消息传递中，为了区分不同通信事件的发送方和接收方，也为了增强后期代码的维护性，系统使用 emit 方法

进行数据通信。关键代码如下：

```
io.on('connect',(socket)=>{
    // 监听连接请求，升级为 WebSocket 通信协议
    // 连接建立
    // 数据通信方法一：send()发送数据对象
    socket.send(obj);
    socket.on('message',(res)=>{
        接收返回信息
    })
    // 数据通信方法二:emit()发送数据对象
    socket.emit(eventName,obj);
    socket.on(eventName,(res)=>{
        接收返回信息
    })
})
```

系统完成初始渲染后，客户端通过 close 方法断开连接。等到服务端开始传输时变数据块时，先由客户端发送连接请求，重新建立连接，再由服务器主动发送数据包。进行查询服务时，由客户端向服务器发送数据查询请求，服务器响应后查找文件，将查询结果返回给客户端。

4.2.2 数据预处理模块设计

数据预处理模块需求分析图如图 4.10 所示，主要负责读取文件、数据文件更新、为之后的数据渲染提供预处理好的初始数据块和时变数据块。该功能模块由系统每个独立的数据服务器完成，使用 Node.JS 进行开发设计。经过预处理之后的数据块经由 WebSocket 通信服务传输给客户端；更新完毕的数据文件保留在服务器中，作为待比较数据参与下一时间点时变数据的预处理。

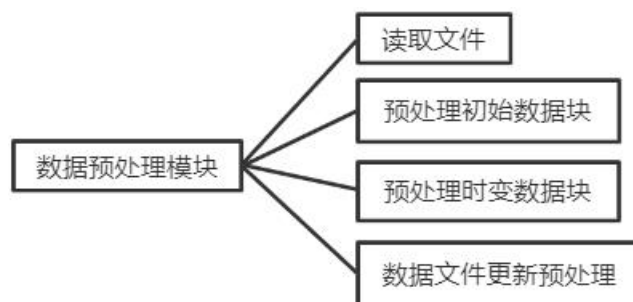


图 4.10 数据预处理需求分析

4.2.2.1 Node.JS

Node.JS 作为一款基于 Chrome 谷歌浏览器 V8 引擎的 JavaScript 运行环境，其执行效率非常高，用于快速搭建响应速度快、易于扩展的网络应用。Node.JS 不仅可以用来编写 JavaScript 客户端运行环境，同时还可以创建服务器，连接数据库，开发交互式终端程序，其实现前后端交互流程如图 4.11 所示。

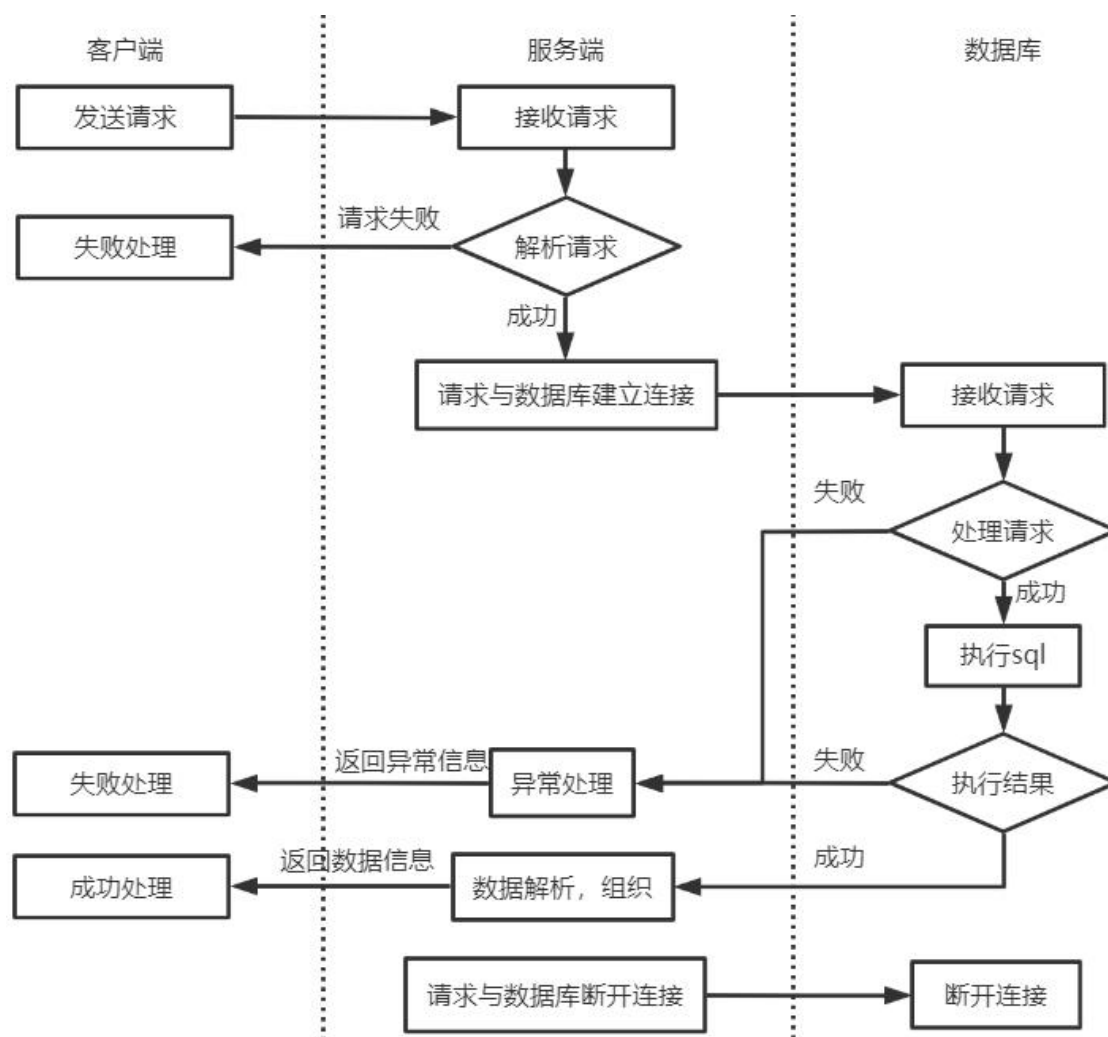


图 4.11 Node.JS 实现前后端交互

Node 的目标是: 旨在提供一种简单的构建可伸缩网络程序的方法^[51]。在 Java, PHP 这类语言中, 新建连接会产生一个新的线程, 每个线程都会分配一定数额的内存。随着线程数量的增多, 分配的内存总量达到系统内存的峰值时, 也就达到了系统增加新线程的瓶颈。如果希望 Web 程序支持更多的用户, 则必须增加新的服务器。Node 为解决这一问题提供了新的思路^[52]: 改变连接到服务器的方式。

Node 不允许使用锁，不直接阻塞 I/O 调用，因此不会产生死锁。Node 通过事件驱动^[53]，每个连接被发送到 Node 引擎进程中运行的事件，取代了每个连接产生一个新的 OS 进程。Node 凭借事件驱动，非阻塞 I/O 模型^[54]变得轻量高效，因此适合在分布式设备上运行数据密集型的实时应用。同时，Node 采用 commonJS 模块规范^[55]，模块定义如图 4.12 所示。每一个 JS 文件作为一个单独的模块，每一个模块都是一个独立的作用域，主要分为：核心模块、第三方模块以及自定义模块。

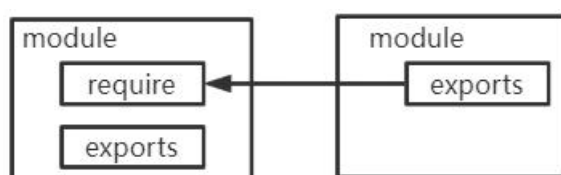


图 4.12 模块化定义

Java, PHP 等语言使用面向对象式的开发设计，而 Node 则使用事件驱动模型的编程思想。事件驱动模型如图 4.13 所示，该模型思想是：为需要处理的事件编写相应的事件处理程序，代码在事件触发时执行，从而释放 CPU 进入睡眠状态，当事件触发时被唤醒，可以更高效地使用 CPU。



图 4.13 事件驱动模型

事件模型包含 3 个对象：事件源、事件对象以及事件处理程序。事件源是事件产生的地方，比如 DOM 元素；当事件发生时就会产生一个事件对象，里面封装了事件类型、时间等信息；事件处理程序就是触发事件时执行的代码。

传统的单线程操作是多个业务请求排队，按序处理。主线程须等待上一个请求的处理结果，再继续执行之后的业务。如果有一个耗时较长的业务，就会造成阻塞。而 Node 中的单线程并非真正的单线程：多个业务按序执行，但主线程并不等待上一请求处理完毕之后再执行下一请求，而是开启其他线程处理 I/O。指令到达主线程后，主线程发现 I/O 操作，将其传给其他线程，获得一个状态之后继续执行下面的业务。I/O 处理完毕后会产一个回调事件，该事件会被放入一个事件队列。进程启动时，Node 会创建一个循环，每一次轮询都会查看事件队列是否有需要处理的事件，如果有就进行处理。这个流程被称作事件驱动机制，

如图 4.14 所示。

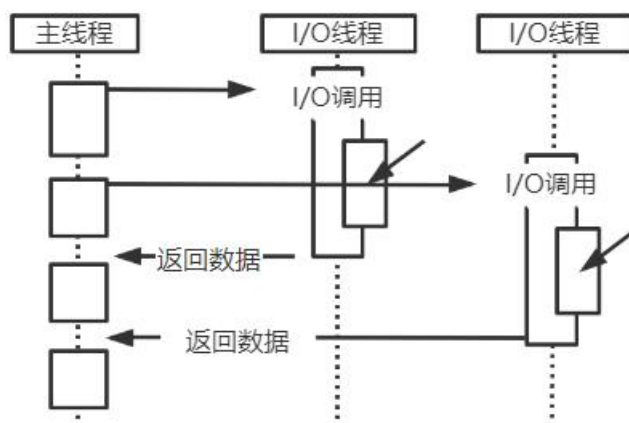


图 4.14 事件驱动机制

4.2.2.2 Node.JS Express 框架

Express^[56]是为 Node.JS 设计的一款 web 开发框架,帮助设计者创建各种 Web 应用以及丰富的 HTTP 工具,快速地搭建一个网站。其核心特性包括:设置中间件响应 HTTP 请求;定义路由执行各种 HTTP 请求动作;通过模板传递参数,动态渲染 HTML 页面。

利用 Express 创建服务器原理如图 4.15 所示,具体步骤如下:

1. 导入 express 模块
2. 创建 Web 服务器
3. 调用 listen 方法监听端口,启动服务器。

```
//1. 导入express模块
const express = require('express');
//2. 创建web服务器
const app = express();
//3. 调用app.listen()函数启动服务器
app.listen(8000, () => {
  console.log('启动服务器');
})
```

图 4.15 创建服务器

Express 使用回调函数中的参数 request 和 response 处理请求和响应的数据^[57], 两者的常用属性分别如表 4.2, 表 4.3 所示。

表 4.2 Request 对象常用属性

属性	描述
app	访问 express 实例
body	获取请求主体
path	获取请求路径
query	获取 URL 查询参数串
route	获取当前的匹配路由
baseUrl	获取路由当前安装的 URL 路径
accepts()	检查请求的 Accept 头的请求类型
get()	获取指定的 HTTP 请求头

表 4.3Response 对象常用属性

属性	描述
app	访问 express 实例
append()	追加指定 HTTP 头
set()	设置 HTTP 头，传入 object 可以一次设置多个头
get()	返回指定的 HTTP 头
json()	传送 json 响应
jsonp()	传送 jsonp 响应
send()	传送 HTTP 响应

在 Express 中，路由指的是客户端的请求与服务器处理函数之间的映射关系，由请求类型、请求的 URL 地址以及处理函数三部分组成。为了便于后期路由维护，Express 提出了模块化路由的概念：具体路由定义在路由对象中，一个路由对象挂载系统所涉及的所有路由，形成一个独立的模块，在运行文件中注册调用。

4.2.2.3 具体流程设计

Node.JS 提供了两个对象：exports 和 require，其中，exports 是模块公开的接口，require 用于从外部获取一个模块的接口。

时变数据预处理具体流程如下：

1. Express 创建服务器。
2. require('fs')导入文件系统，用于操作文件。fs.readFileSync()方法支持异步读取文件，以 Buffer 数据流或者 String 字符串的形式获取数据。
3. 结合数据划分规则划分数据块。

4. 遍历数据块，不同时间相同位置的数据差值计算，若结果大于等于时变阈值，则作为变化数据筛选出来，组织为时变数据。

5. 重复步骤 4，直至数据块遍历完毕。

时变数据预处理设计关键代码如下：

```
const fs = require('fs');
var datasOld = fs.readFileSync(OldFile);
var datasNew = fs.readFileSync(NewFile);
// 数据划分
for data in datasOld
    dataBlock.push(data);
    DataBlocks.push(dataBlock);
// 遍历数据块
for dataBlock in DataBlocks
    for data in dataBlock
        // 差值计算
        if (data - dataOld >= 0.1)
            // 组织时变数据
            changeData = (i,j,k,value);
            changeDataBlock.push(changeData);
            changeDataBlocks.push(changeDataBlock);
```

数据文件更新预处理具体流程如下：

1. require('fs')导入文件系统，fs.readFileSync()方法获取比较数据和待比较数据。

2. 遍历数据，计算差值。差值大于等于阈值，待比较数据中该位置的数据替换为比较数据中相同位置的数据；差值小于阈值，待比较数据保持不变。

3. 利用 fs.writeFile()方法将更新后的数据写入文件，数据文件更新预处理完成。

关键代码如下：

```
const fs = require('fs');
var datasOld = fs.readFileSync(OldFile);
var datasNew = fs.readFileSync(NewFile);
// 遍历数据
for data in datasOld
    // 计算差值
    if (dataNew - data >= 0.1)
```

```
// 数据替换
data = dataNew
file = fs.writeFile(fileName,datasOld);
```

4.2.3 数据渲染模块

高分辨率数据可视化面临的首要问题就是庞大的数据量，如何实现数据快速分类、快速传输以及高效渲染，展现流畅的动态渲染效果，一直是数据可视化领域研究的重点。随着 LOD 算法的首次提出，金字塔模式和基于 LOD 的动态调度算法成为了实现数据动态可视化的主流方案。然而，硬件设备的发展解放了屏幕大小，数据分辨率的提高也使得其可视化图层需要更大的屏幕作为载体进行展示。金字塔模式、LOD 动态调度算法对于已经渲染好的图层按时间顺序加载展示效果良好，但是，其加载的数据可视化图层仍然需要经过渲染操作，后期需要转换数据格式，发布到服务器上，再经由客户端请求，才将某一范围的图层发送给前端进行加载。其相比于数据直接渲染，降低了渲染实时性，不适合实时性较高的应用场景。同时，在该模式下，全域视角下展示的是低分辨率数据的可视化图层，只有照相机与三维地球模型的距离达到一定程度之后，才加载部分范围的高分辨率数据可视化图层。面对当前环境下的大屏幕，较低分辨率数据可视化图层在展示时边缘锯齿显得尤为突出，使用户难以适应，由此限制了屏幕大小。所以，为实现全域高分辨率海洋标量数据的实时动态可视化渲染以应对实时场景应用，本文研究设计了基于时变阈值与时变数据预处理的数据可视化方法，并基于时变数据设计了数据渲染模块。

数据渲染模块需求分析图如图 4.16 所示，其首要任务就是接收来自服务端的数据块，分析转换初始数据的坐标数据格式，利用 Cesium 封装的 WebGL 渲染器渲染要素值，将渲染图层加载到三维地球模型上；计算命令标识，更新渲染命令，完成时变渲染。

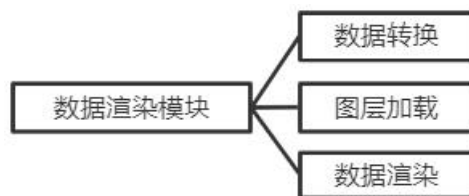


图 4.16 数据渲染模块需求分析

Cesium 中支持的坐标系有 WGS84 地理坐标系和笛卡尔空间直角坐标系^[58]，两种坐标系分别如图(a)，图(b)所示。但是，Cesium 中并没有实际对象应用的 WGS 地理坐标系，一般都是转换为弧度（Cartographic）或者笛卡尔空间直角坐

标系 (Cartesian3)。若采用弧度转换, 利用 `new Cesium.Cartographic()` 创建弧度对象, 参数包括: 经度、纬度以及高度。如图 4.18 所示, 目标弧度 = $(180/\pi) \times$ 经纬度角度。

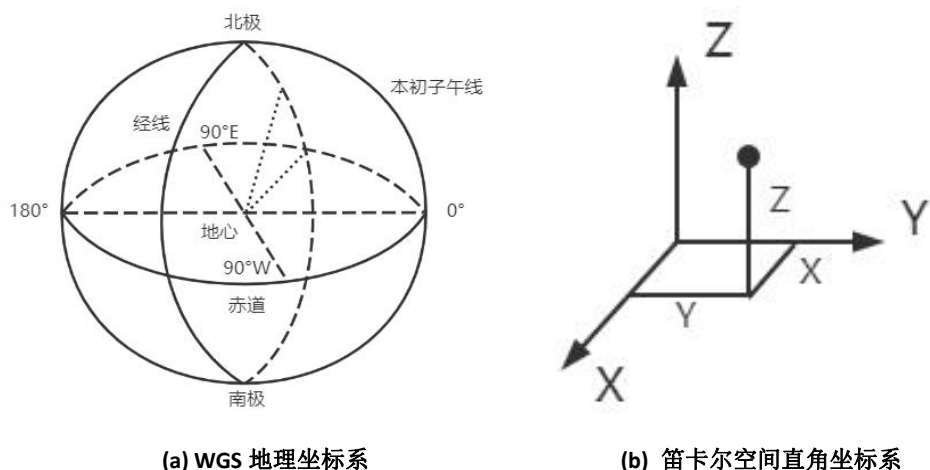


图 4.17 坐标系

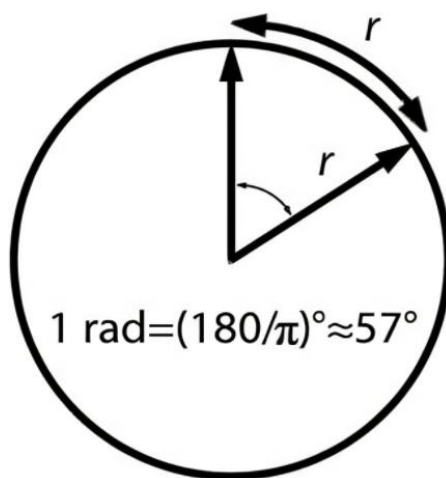


图 4.18 角度、弧度转换

由于计算机不方便直接使用经纬度绘图, 因此, 需要通过笛卡尔空间直角坐标系建立空间内的点与有序数组之间的联系。笛卡尔空间直角坐标系由互相垂直、长度相同且交汇于同一点的三条数轴组成, 中心交汇点被称为坐标系原点, 三条数轴分别叫做 x 轴 (横轴), y 轴 (纵轴) 以及 z 轴 (竖轴), 统称为坐标轴。x 轴和 y 轴通常配置在水平面上, z 轴作为铅垂线, 坐标轴正方向符合右手定则。

系统进行数据渲染之前, 将地理坐标系中的经纬度坐标转换为笛卡尔空间直角坐标系每个坐标轴上所对应的分量: 使用 `new Cartesian()` 创建笛卡尔坐标系对象; 利用对象中的 `fromDegrees` 方法进行数据转换, 参数包括: 经度, 纬度以

及高度。坐标转换完成后,利用插件将该点的要素信息值转换为 RGBA 颜色值,与坐标值一起存入缓存器,利用 Render 模块调用 GPU 进行数据渲染。根据每个顶点的位置坐标绘制顶点位置,并装配为三角形图元,经过光栅化和片元处理后,两个三角形图元合并为一个像素,利用要素值转换得到的 RGBA 颜色值填充像素,最终完成一个像素点的绘制,所有像素点组成某一时刻的全域海洋要素信息渲染图层。时变渲染使用新的要素值转换得到的 RGBA 值重新渲染像素点。Cesium 的 Render 模块将上述步骤进行封装,最终通过 new Cesium.DrawCommand() 生成渲染命令对象,加入命令列表。系统每申请一个渲染帧,循环执行列表中的渲染命令,完成数据渲染。

Cesium 自 Viewer 对象创建完成开始,实例化 CesiumWidget^[59]对象,在其内部定义了一个定时器,时刻不停地请求渲染帧,调用 Render 模块,渲染包括天空、大气层以及加载在 Cesium 上的各种图层、实体、数据。当需要渲染的命令过多时,频繁的渲染不仅会造成庞大的系统开销,在转动地球模型时也会变得卡顿。因此,系统重新定义了渲染帧的请求进程,只有满足特定情况下,即只有每一个数据块渲染完成或照相机视角改变时才会请求新的渲染帧,从而解决系统的卡顿问题。

渲染模块渲染数据具体操作流程如下:

1. 遍历数据包,使用 Cesium.Cartesian3.fromDegrees() 将经纬度坐标转换为笛卡尔空间直角坐标系中对应坐标轴上的分量。
2. 要素值数据转换为 RGBA 值,使用 new Cesium.Buffer.createVertexBuffer() 创建顶点缓冲区存储顶点位置和颜色。创建索引数组,规定渲染对象顶点的渲染顺序。
3. 整合顶点位置、颜色、法线等信息生成属性数组。利用 new Cesium.VertexArray() 创建顶点数组,存储顶点位置、颜色以及索引。
4. 使用 new Cesium.DrawCommand() 创建渲染命令,加入 commandList 命令列表。系统请求渲染帧,遍历命令列表,依次执行渲染命令,实现初始渲染。
5. 计算命令标识,定位渲染命令,查找更新位置,转换要素值,更新渲染命令,实现时变渲染。

4.3 系统功能实现

作为用户与系统之间沟通的桥梁,完善的交互功能可以给予用户良好的使用体验。系统实现的功能如图 4.19 所示,系统接收用户不同的指令,并将这些指令正确传达给对应的事件接收器,触发事件监听函数,完成响应事件,从而保证各种功能的正常运行。主要功能除了基本的图层切换、时变图层渲染之外,还应包括特定点要素信息查询、自定义范围划定、不同海洋要素值类型切换等一系列

功能。同时，系统利用 Echarts 绘制包括折线图、饼图、柱状图等各种图表，帮助用户分析数据变化情况，更快掌握数据变化规律。

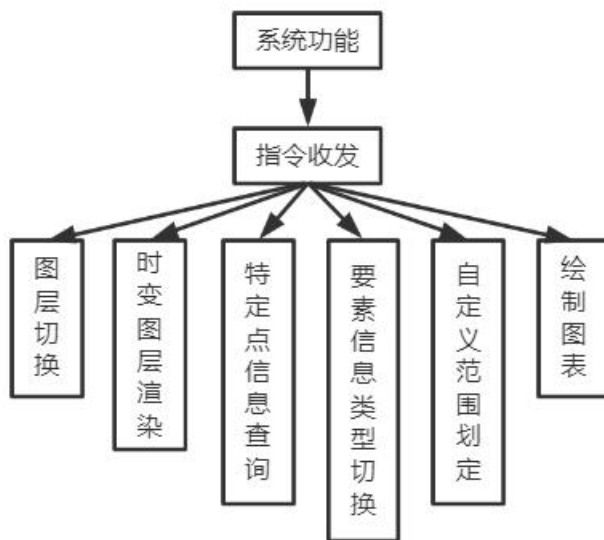
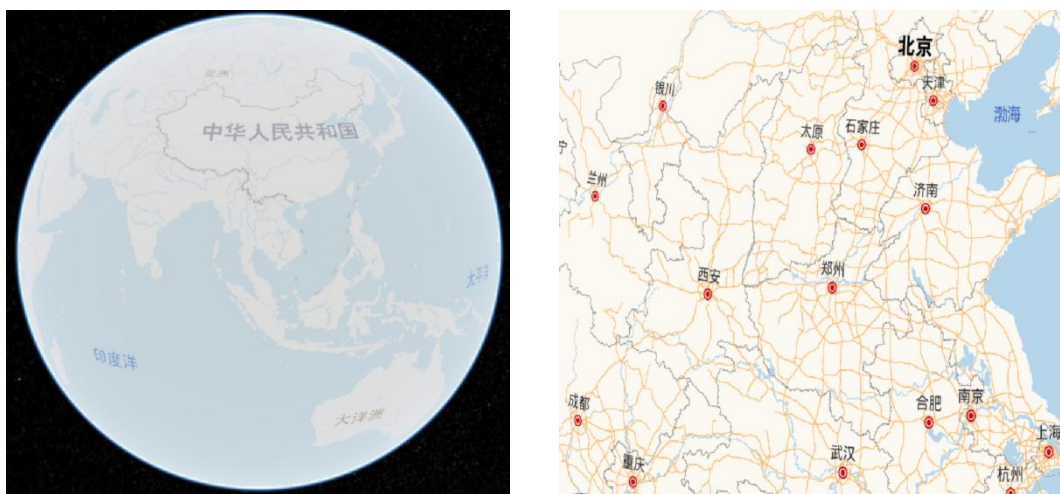


图 4.19 系统功能实现

4.3.1 图层切换

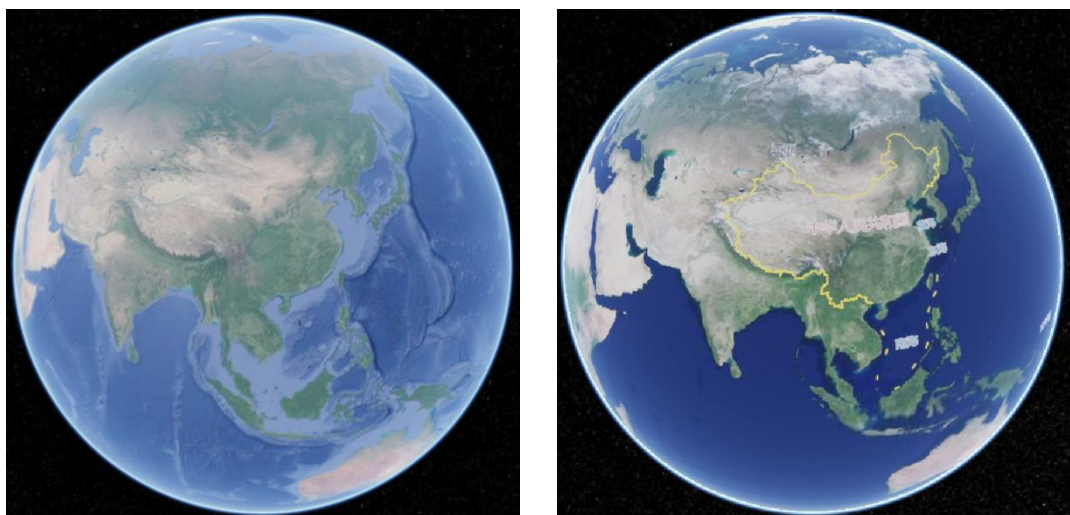
Cesium 自身提供多种图层^[60]供用户切换，除了使用 ArcGIS、谷歌、高德等公司制作的图层，Cesium 也支持加载 Geoserver 发布的图层。Viewer 对象中的 imageryProvider 属性控制各种图层的加载，加载各种图层效果如图 4.20 所示。Cesium 不仅支持各种在线地图的加载，同时也支持离线地图的加载。使用类似太乐地图下载器之类的软件下载不同层级的图层，将图层切割成瓦片进行存储，Cesium 会根据照相机距离地球模型的距离调用不同层级的瓦片。



(a) 高德地图图层



(b) ArcGIS 在线地图图层以及天地图图层



(c) 谷歌地图图层和离线地图图层

图 4.20 图层

Cesium 除了支持图层加载服务，同样支持地形加载。通过 `new Cesium.TerrainProvider()` 创建地形对象，对象赋值给 Cesium 中的 `terrainProvider` 属性，最终效果图如图 4.21 所示。



图 4.21 地形

4.3.2 时变图层渲染

图层加载以及地形加载只是 Cesium 可以提供的基础服务，目的是利用不同的图层作为底层满足不同用户的需求和喜好。

系统客户端的主要作用是为了展示可视化图层、图表以及各种响应结果，并不直接接受用户指令。用户所有手动操作均在手机端或者 pad 端上进行。用户在移动端的各种操作指令传输给客户端，触发响应事件函数，实现客户端操作同步响应。移动端界面和客户端界面如图 4.22 所示。

移动端允许用户完成选择图层、要素值数据类型等前期准备。用户根据自己的需要选择基本图层以及要素值类型，选定图层、要素值类型的特征值会经由 MQTT 通信协议传递给客户端。客户端接收图层特征值之后直接在主服务器上请求对应图层进行转换；通过 WebSocket 协议传输要素值类型特征值告知数据服务器准备对应类型的数据块。用户可以在移动端转动地球从而改变视野范围，客户端同步响应照相机位置变化事件，执行事件函数，在屏幕上展示地球转动效果。

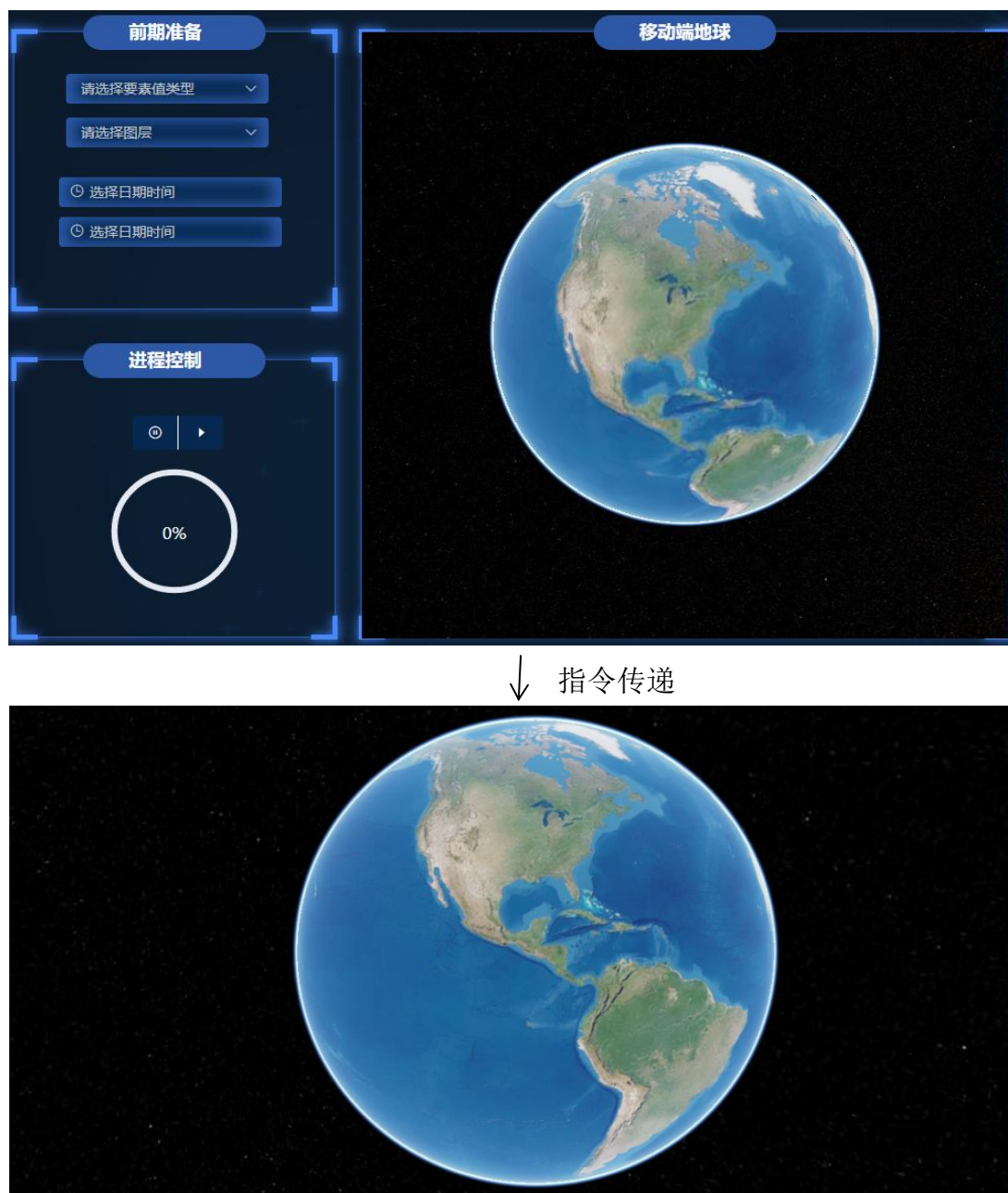


图 4.22 移动端界面（上）客户端界面（下）

EMQ X 提供的免费公共服务器基于 EMQ X 的 MQTT 物联网云平台创建，支持基于 WebSocket 的 MQTT 协议，TCP 端口是 1883，WebSocket 端口 8083 用于普通连接，状态码显示为 101 时表示 MQTT 通信成功建立。此时，移动端、客户端之间可以进行同步操作。

如图 4.23 所示，用户选择高德地图作为底层图层，选择海洋温度作为时变可视化数据类型。用户转动三维地球，调整到自己想要查看要素值变化的位置，前期准备完成。

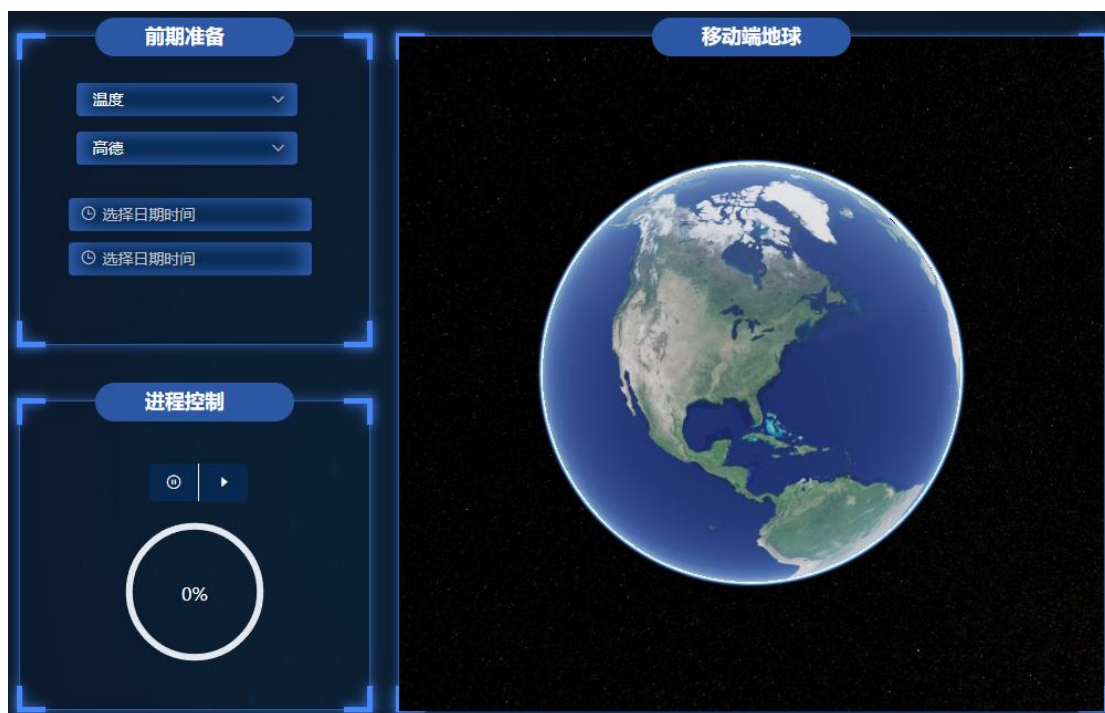


图 4.23 移动端前期准备

用户在移动端点击三维地球下方的“开始”、“结束”即可控制可视化进程。点击‘开始’按钮，建立客户端与服务器之间的 WebSocket 通信，客户端通知服务器推送数据。服务器收到信号，按序传输已经预处理好的初始数据块，交由客户端渲染。经过坐标转换、要素值颜色转换，每一个初始数据块都对应一个顶点数组，生成一条渲染命令，存储在命令列表中，在下一个渲染帧中被渲染为可视化图层。海洋温度数据初始渲染图层如图 4.24 所示。

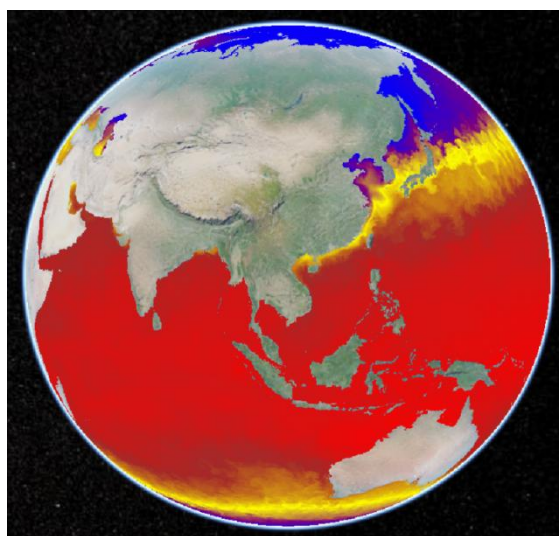
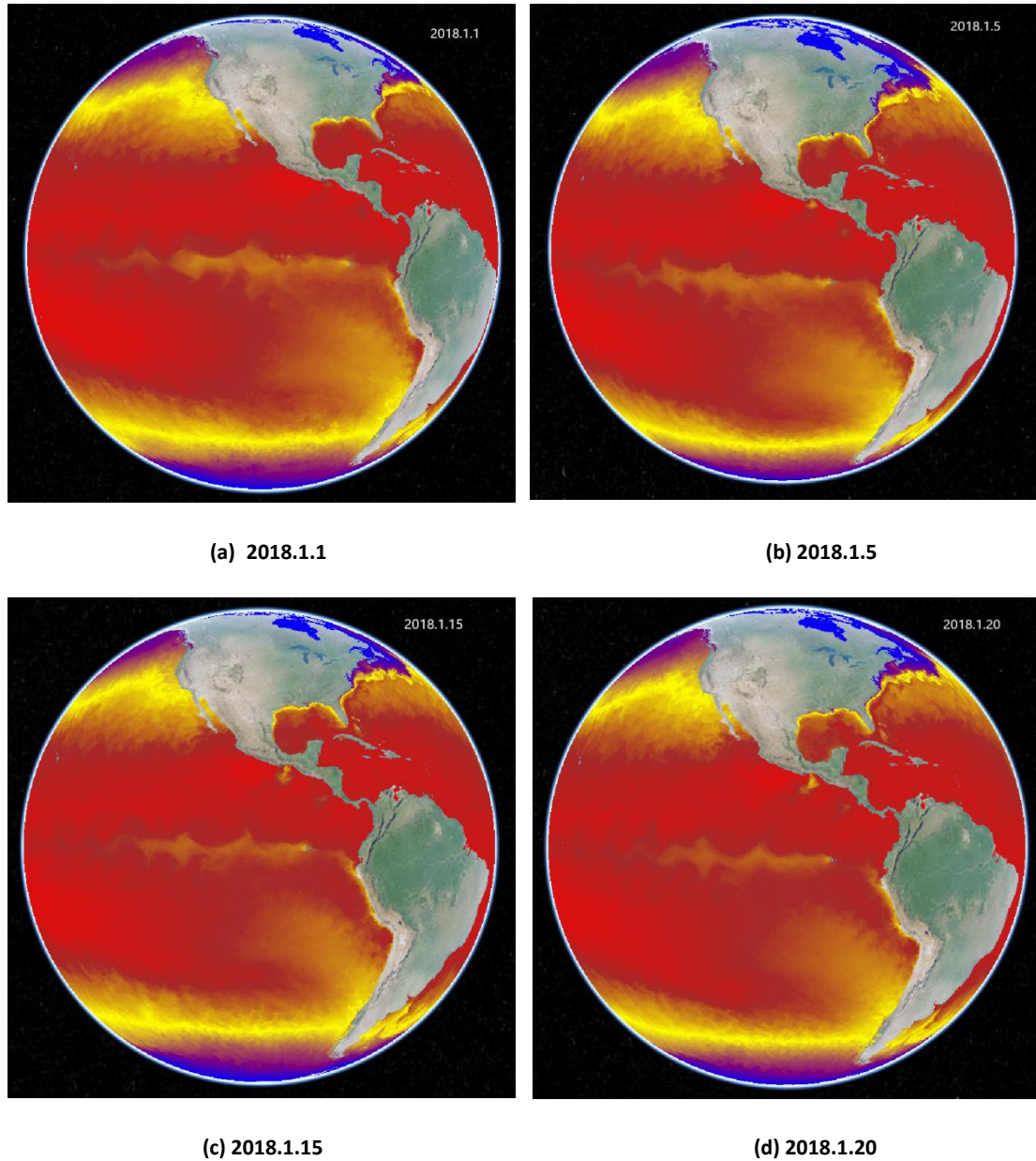


图 4.24 海洋温度数据初始渲染图层

如图 4.25 所示，选取不同时间节点的全域海洋温度数据渲染图层。系统结合时变阈值，组织时变数据，通过计算得到的命令标识和更新数据位置标识快速、准确定位待更新的渲染命令以及要素值，实现渲染命令按需更新，在不影响观察效果的前提下提高数据渲染效率。



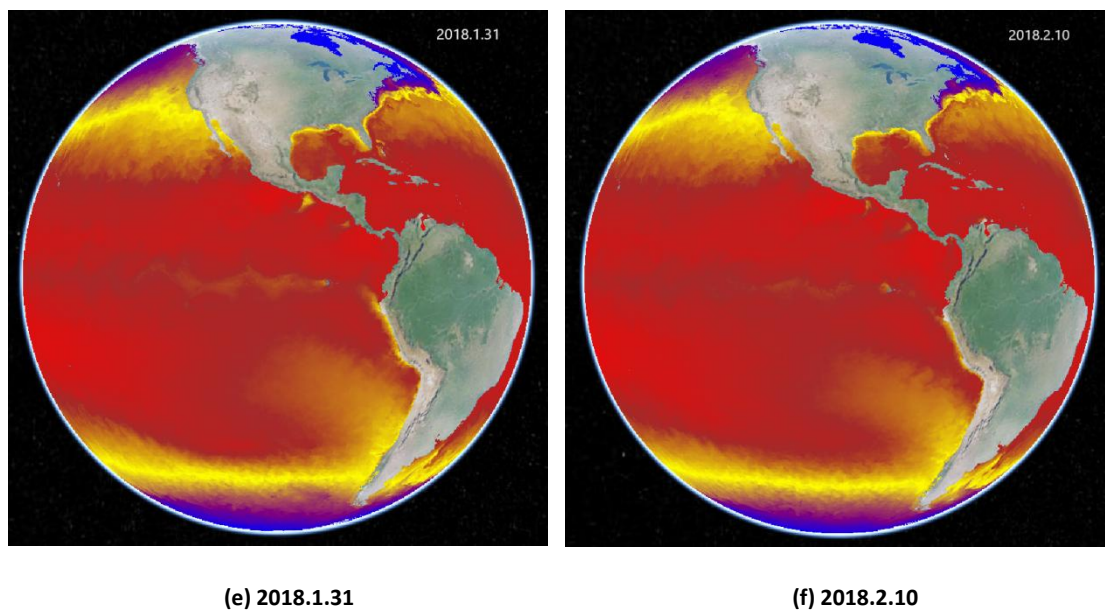


图 4.25 不同时间点温度数据渲染图层

用户如果想要切换渲染要素值类型，在移动端点击‘结束’按钮结束当前渲染进程。客户端收到指令，断开 WebSocket 通信连接，清空命令列表中的渲染命令。用户根据左侧选择器选择想要渲染的要素值类型以及基础图层，点击“开始”按钮向客户端发送渲染指令。客户端重新恢复与服务器之间的连接，请求渲染所需类型数据，开始渲染进程。海洋表面盐度的渲染图层如图 4.26 所示。

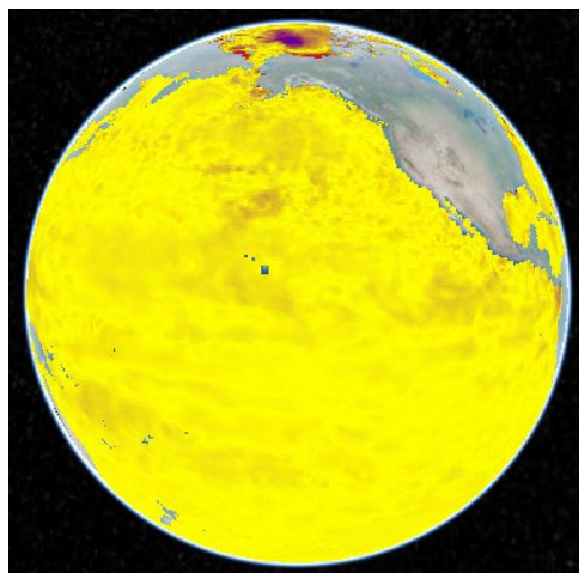


图 4.26 海洋表面盐度图层

4.3.3 数据查询

除了基本的渲染、进程控制、要素值类型切换等功能，系统还提供了特定点要素数据查询功能、自定义范围划定功能以及图表绘制功能。

如图 4.27 所示，用户在移动端选择想要查看要素信息的位置，在移动端生成带有经纬度位置属性的对象，触发数据查询事件，通过 MQTT 通信传递给客户端。客户端根据经纬度计算得到待查询位置所属服务器的标号以及在文件中对应的行下标和列下标，创建查询对象，对象属性包括服务器标号、行下标、列下标以及日期标号。服务器标号用于确定与客户端进行数据交互的数据服务器，日期标号确定待查询数据所在的文件。服务器根据数据所在的行下标和列下标找到对应位置的要素值，作为响应数据传给客户端。由于 Cesium 自带的 label 控件文字清晰度较差，所以在查询位置创建一个元素框，显示查询位置的经纬度数据和要素值信息。

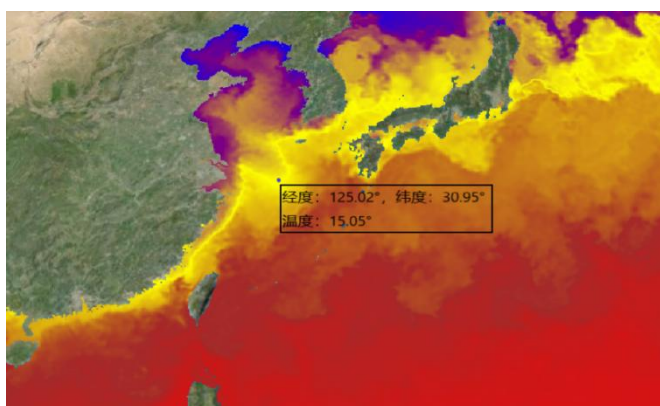


图 4.27 特定点数据信息查询

4.3.4 自定义范围划定

自定义范围划定功能如图 4.28 所示。用户如果想要拉近照相机与模型之间的距离，查看特定范围的渲染效果，只需在移动端选择一个初始点 `startPoint`，再选择一个终点 `endPoint`，两个点的经纬度数据整合为一个对象传递给客户端。客户端以 `startPoint` 作为自定义范围的左上角顶点，`endPoint` 作为右下角顶点创建一个矩形，通过 Cesium 自带的 `camera` 对象飞行到可以观察到该矩形范围的最佳位置，以此替代用户自己利用鼠标、滑轮调整照相机的位置，方便用户更快获取自身所需要的信息，掌握变化规律。



图 4.28 自定义范围划定

4.3.5 图表绘制

绘制图表功能包括绘制折线图、饼图进行数据分析。如图 4.29 所示，用户在移动端转动地球模型，选择一个目标点，利用系统上方的时间日期选择器选择起始时间点和结束时间点规定时间范围。



图 4.29 图表绘制移动端操作

点击确定按钮，触发绘制图表事件。客户端分析计算目标位置得到对应的要素值所在的服务器标号以及所在文件中的行下标和列下标；计算起始日期和结束日期之间的天数间隔，找到每一天对应的数据文件，在文件中找到正确位置的要素值。服务器将要素值和排位序号发送给客户端，由客户端排序得到要素值数组。

Echarts 作为一款开源的数据可视化工具，兼容当前绝大部分浏览器，提供多种直观、可交互、高度个性化定制的数据可视化图表。系统引入 Echarts 方法库，自定义图表属性，以温度数据为例，在可视化图层两侧生成三张目标点自定义日期范围温度变化图表，如图 4.30 分别是摄氏度温度变化折线图、开尔文温度变化折线图和开尔文温度变化柱状图。在左侧标明数据类型、位置和日期属性。系统通过提供多类型的不同单位的温度变化图表，帮助用户快速掌握数据变化规律。

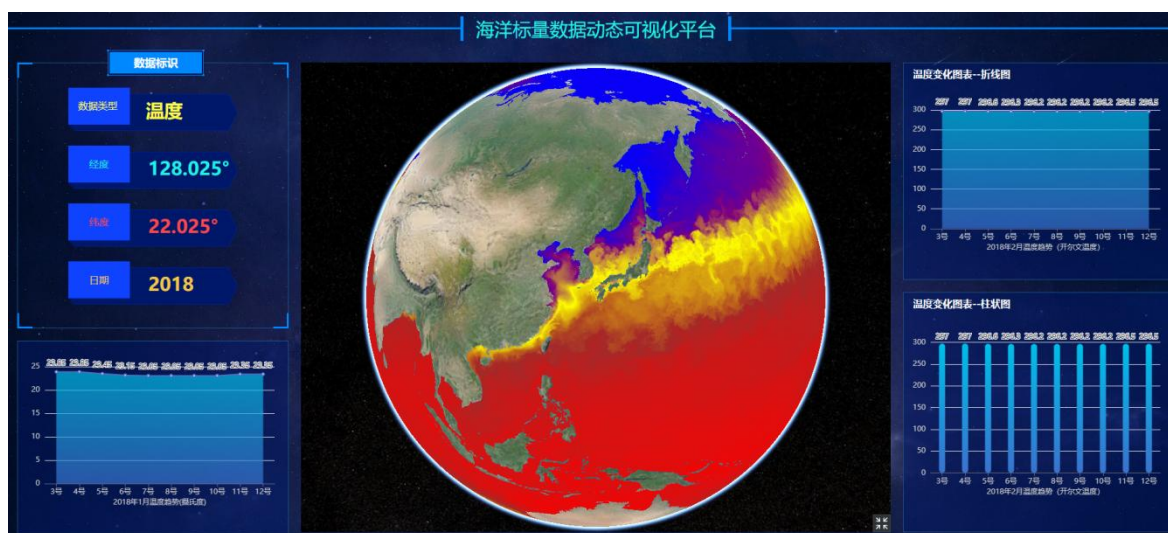


图 4.30 目标点温度变化图表绘制

4.4 本章小结

本章主要介绍了本文设计实现的系统的体系架构、组成模块以及所完成的各种功能。系统体系架构方面主要介绍了系统分布式架构和系统层级。分布式架构方面，主要介绍了各部分的组成以及每部分所承担的功能，分析了移动端、客户端和服务端之间的数据通信方式。系统层级方面主要介绍了系统不同层级功能和它们之间的逻辑关系。系统模块分为：数据通信模块、数据预处理模块、数据渲染模块。数据通信模块主要介绍该模块需要满足的需求，借助代码分析系统使用的两种协议是如何建立连接的，数据是如何传递的；数据预处理模块主要分析模块需求，简单介绍了 Node 和 Express 框架，具体分析了预处理流程和实现方法；数据渲染模块主要分析功能需求，介绍了坐标类型转换和渲染命令的生成和更新。系统功能主要介绍该系统应满足的功能需求，除了基本的时变数据渲染功能外，还包括定点查询、划定范围和生成图表，介绍了功能的实现方式和操作流程。

第五章 系统性能测试

本章主要通过一系列性能对比说明本文研究系统的优势以及针对以往数据渲染所做出的改进,从而体现本文提出的数据可视化方法的使用价值与意义。对比不同分辨率数据渲染图层效果,突出渲染高分辨率数据的重要性、对掌握海洋标量要素信息的重要性;对比传统的 Ajax 通信与 WebSocket 通信传输数据所用的时间和性能消耗,突出 WebSocket 通信优势;对比单台服务器和分布式架构处理数据的速度,对比渲染是否经过预处理优化的时变数据所需时间,突出本文研究的系统使用的分布式架构和提出的数据预处理方法的优势和使用价值。本章利用真实数据、系统实验绘制的数据渲染图层、实验数据对比图表佐证本文设计系统的优势。

5.1 不同分辨率渲染图层效果对比

海洋高分辨率数据动态可视化渲染不仅可以直观表达更为精确的海洋要素信息,帮助用户掌握更为准确的海洋信息变化规律,而且,在基于大屏展示时,图层渲染效果要优于低分辨率数据渲染图层。如图 5.1 所示,分辨率为 0.05° 的海洋温度数据渲染图层在视觉效果上要优于分辨率为 0.4° 的海洋温度数据渲染图层。由于大屏分辨率较高,而利用 GPU 渲染的图层是基于像素组成的,每个像素点是由两个三角形图元组成的矩形,放大过程中边缘锯齿愈发明显,影响用户的视觉体验。这也是低分辨率数据可视化图层限制屏幕大小的原因。同时,高分辨率数据可视化图层在要素数据和地形数据的边缘处理上效果较好,海洋面积几乎被全部覆盖,既没有空间遗漏,也没有地形被覆盖填充,总体视觉效果上优于低分辨率数据渲染图层。

高分辨率数据可以提供更加详细的海洋要素信息。每个渲染点所体现的是某一空间范围内的要素信息:数据分辨率为 0.4° ,每个渲染点的要素值代表周围经纬度范围为 0.4° 空间的要素信息;数据分辨率为 0.05° ,渲染点的要素值代表周围经纬度范围为 0.05° 空间的要素信息。因此,数据分辨率越高,像素点的数据量就越多,对海洋面积地切分就越精细,在不同时间点所提供的要素信息越详细、精确,为之后的研究提供更有价值的参考。

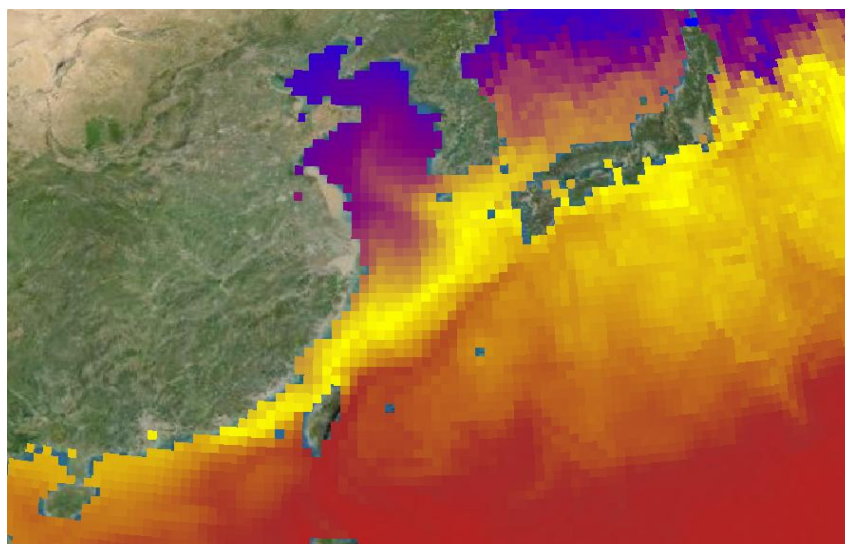
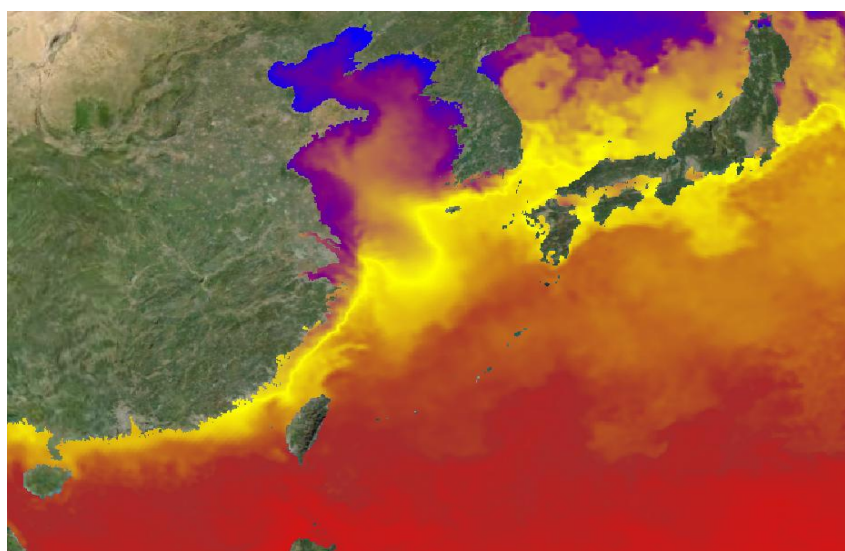
(a) 分辨率 0.4° (b) 分辨率 0.05°

图 5.1 不同分辨率数据渲染图层

5.2 数据通信性能对比

系统基于 WebSocket 通道，使用 MQTT 协议进行通信，在移动端和客户端之间准确传递不同指令，实现同步操作。客户端和服务端之前同样使用 WebSocket 协议实现双向数据通信。与传统的 Ajax 异步通信相比较，WebSocket 通信协议除了支持双向通信，可以保持长时间稳定连接、较小的宽带开销也是其重要优势。

本研究使用谷歌浏览器开发者模式中的网络监测工具 Network 对传统 Ajax 轮询方式和 WebSocket 协议通信方式产生的网络消耗进行对比，两者在传输相同

数据的情况下，根据图表直观展示对比结果。如图(a)所示，传统 Ajax 轮询方式由客户端逐次向服务端发起请求，每一次数据传输都需要客户端和服务端完成一次‘请求/响应’操作。使用 Ajax 轮询技术，查看监测工具上显示数据传输消耗的时间；相较于 Ajax 技术，图(b)使用的 WebSocket 通信协议实现前后端数据交互时，只需要客户端向服务端发送一次数据请求，将原有的 HTTP 协议升级为 WebSocket 协议，通道建立完成之后，不再产生网络消耗。传输相同数据，监测工具显示用时。两种方式经过对比发现，WebSocket 通信方式传输数据的时间以及网络消耗低于 Ajax 轮询方式，性能较好，减轻服务器负担。

☐ ajax	200	xhr	jquery.js:2	245 B	2 ms
☐ ajax	200	xhr	jquery.js:2	245 B	2 ms
☐ ajax	200	xhr	jquery.js:2	245 B	1 ms
☐ ajax	200	xhr	jquery.js:2	245 B	4 ms
☐ ajax	200	xhr	jquery.js:2	245 B	4 ms
☐ ajax	200	xhr	jquery.js:2	245 B	3 ms
☐ ajax	200	xhr	jquery.js:2	245 B	3 ms

(a) ajax

Data	Length	Time
42["message",{"data":99}]	25	11:01:35.442
42["message",{"data":99}]	25	11:01:35.443
42["message",{"data":99}]	25	11:01:35.443
42["message",{"data":99}]	25	11:01:35.444
42["message",{"data":99}]	25	11:01:35.445
42["message",{"data":99}]	25	11:01:35.445
42["message",{"data":99}]	25	11:01:35.446

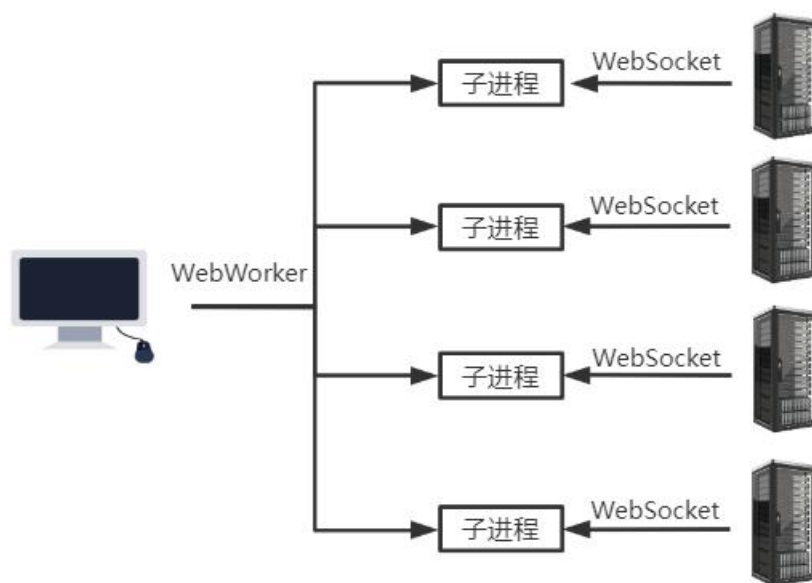
(b) websocket

图 5.2 不同通信方式性能对比

5.3 数据渲染性能对比

本系统使用分布式架构，由 4 台数据服务器独立获取最新海洋标量要素数据、进行时变数据预处理和数据文件更新预处理、存储数据文件，每一台服务器硬件相同，操作系统为 64 位 win10 系统，处理器为 Intel Core i5，内存 16 G，显卡为 NVIDIA GeForce GTX 1050 Ti。如图 5.3(a)所示，分布式架构将全域工程按照经纬度范围分解为多个子工程，分散发布到不同的服务器上。每台数据服务器各自连接客户端利用 WebWorker 创建的一个子进程，每个子进程接收预处理数据并传递给主进程，从而实现时变数据并发读取、预处理、传输、接收和存储，利用数据并发性节省性能开销和处理时间。相反如图 5.3(b)所示，普通架构下，只有一台服务器接收最新数据文件，单独读取最新时间点下全域范围内的要素数据，独自预处理组织时变数据，存储当前时间下所有数据，与客户端建立通信连接之

后，全部数据块依次传输给前端，极大消耗服务器性能，占用大量时间。



(a) 分布式架构



(b) 普通架构

图 5.3 系统架构对比

如图 5.4 展示了系统使用不同架构情况下数据读取、预处理以及传输所用时间对比。通过对比相同环节下使用不同架构的系统完成任务所消耗的时间，可以清楚地观察到，本系统使用的分布式框架在读取、预处理、传输环节中消耗的时间要少于普通架构，减少了性能消耗，提高数据处理和发送的效率。

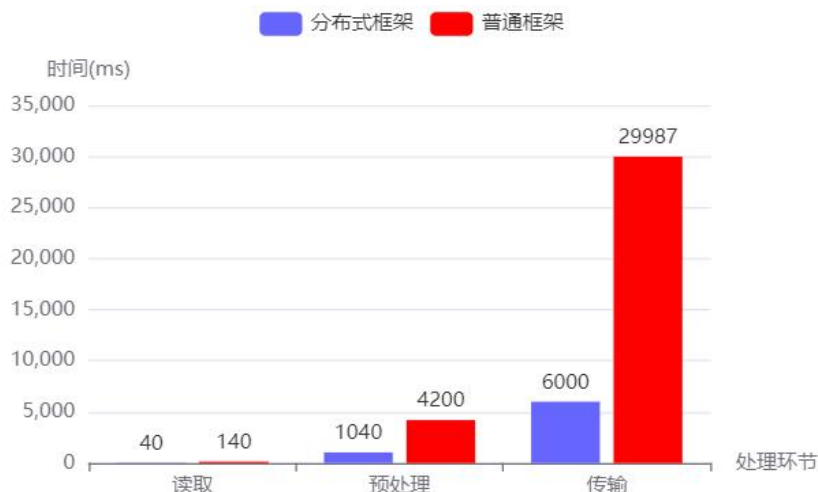
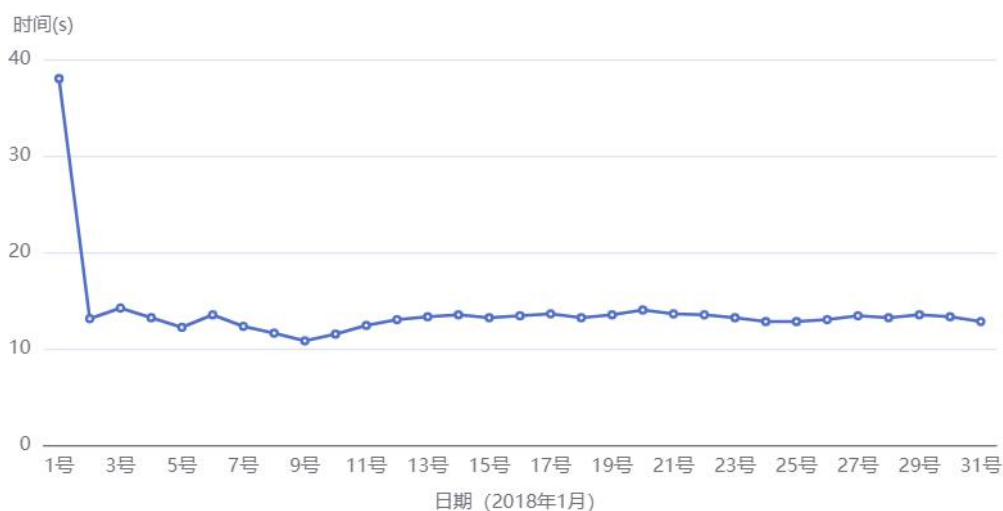
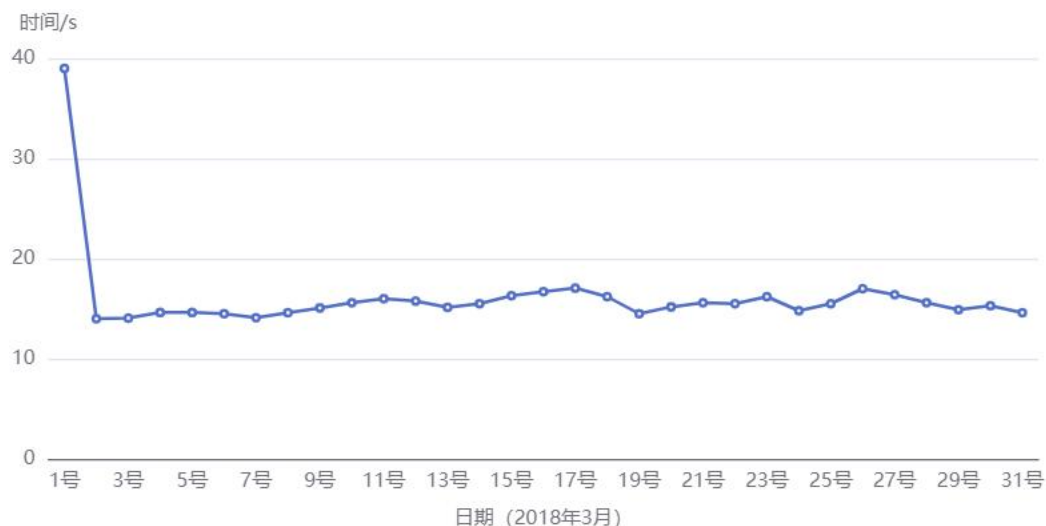


图 5.4 不同处理环节消耗时间对比

如图 5.5(a)所示, 选取 2018 年 1 月 1 日至 2018 年 1 月 31 日, 为期 31 天的海洋温度数据作为实验数据进行时变渲染, 经过多次实验, 计算平均值得到时变渲染所需时间变化图。1 月 1 日时间点下为初始数据, 需要经过初始渲染, 服务器将经纬度数据和要素值结合打包成数据包发送给前端, 经过数据转换, 生成对应的渲染命令加入渲染队列, 启动渲染进程完成数据渲染。初始渲染过程只是简单的数据结合和数据转换, 没有经过时变组织优化。自 1 日起, 时变渲染所需的时变数据都是提前经过时变预处理组织优化得到的, 时变渲染时间与初始渲染时间相比明显减少, 时变效率得到明显提升。2018 年 3 月不同日期时变渲染所需时间变化如图 5.5(b)所示, 用于佐证本文提出的时变可视化方法对于提升时变渲染效率的有效性。



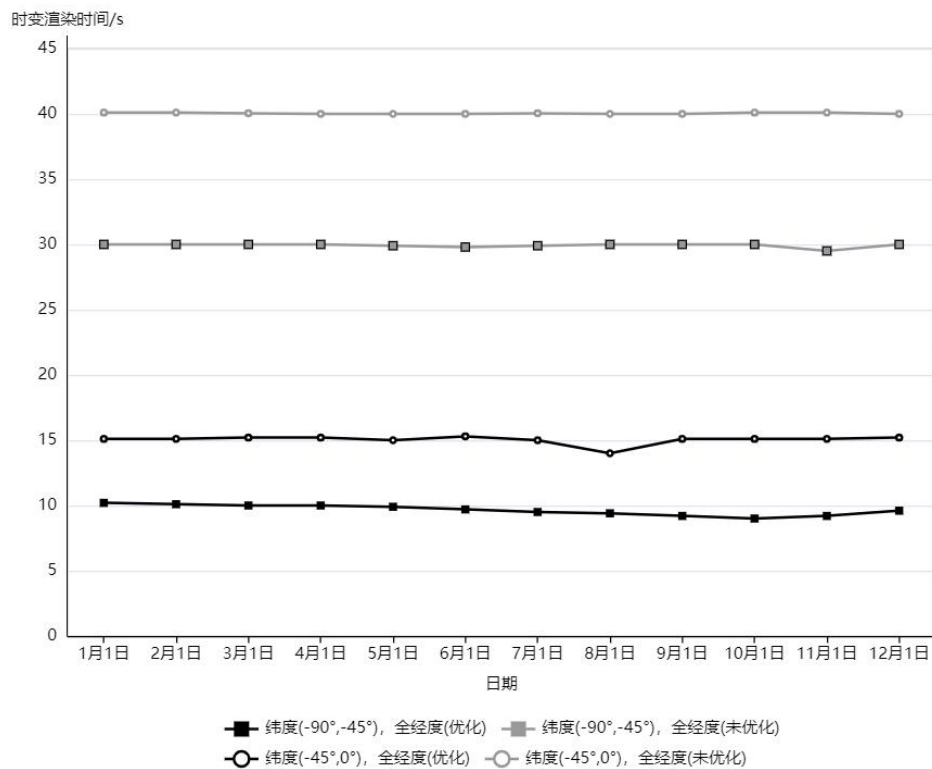
(a) 2018 年 1 月



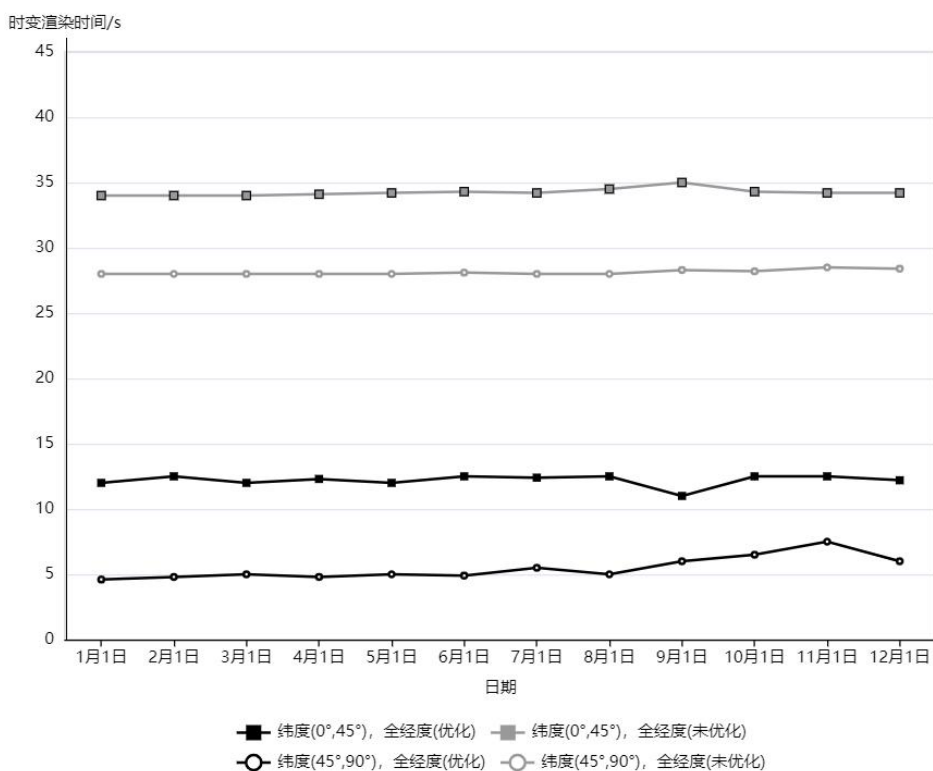
(b) 2018 年 3 月

图 5.5 时变渲染消耗时间

图(a)和图(b)展示了不同经纬范围内以一个月为时间跨度, 经过组织优化的时变数据与未经过组织优化的时变数据, 两者时变渲染所需时间对比。该实验初始渲染的经纬度不由客户端计算, 由服务器读取位置数据传给客户端。如图所示, 时变数据经过组织优化之后, 时变渲染所用时间明显减少, 数据渲染效率明显提高, 且在不同时间段内渲染时间有着较为明显的变化。本文研究的系统所使用的时变渲染方法结合时变阈值筛选变化数据, 在不影响数据可视化效果的前提下去除非必须渲染数据, 减少每次时变渲染的数据量, 减少渲染进程的压力。为了实现并发处理组织数据, 本文系统使用分布式架构, 使初始渲染过程中创建的渲染命令并非是有顺序的。未经过组织优化的时变数据无法正确匹配到渲染命令, 导致每次时变渲染需要对全域数据全部重新渲染, 每次传递数据都需要传递经纬度冗余数据, 从而保证渲染对象加载到正确位置, 影响更新渲染效率。时变数据经过优化之后, 通过命令标识准确定位渲染命令和命令中待更新数据的位置, 更新数据得到新的渲染命令, 完成时变渲染。优化方法在数据传输过程中去除经纬度冗余数据, 减少数据传输量, 提高数据传输速度; 不再对全域数据进行全部更新处理, 只更新时变数据, 实现渲染命令的按需更新, 加快数据渲染速度, 提升时变更新效率, 所需时间的快慢取决于变化数量。



(a) 纬度 (-90° , 0°) , 全经度时变渲染用时差异



(b) 纬度 (0° , 90°) , 全经度时变渲染用时差异

图 5.6 不同范围时变用时差异对比

经过实验分析,本文系统使用的数据通信方式、采用的系统架构、提出的基于时变阈值的时变数据可视化方法,相比之前的系统在数据组织、传输和渲染方面都有了较大提升。使用分布式架构不仅提升了数据组织和传输效率,在数据查询以及数据存储方面也有提高:缩小数据查询的范围,提高数据检索速度;不同经纬度范围数据使用多台服务器存储,减少数据文件的大小,节省服务器存储空间。本文所提出的数据可视化方法实现了渲染命令的按需更新,提高时变渲染的速度,相比于主流的 LOD 动态调度算法提高了数据渲染的实时性,更适合实时性较强的场景。针对全域高分辨率数据渲染解放了屏幕大小,使得屏幕大小不再受到低分辨率数据可视化图层的约束,可连接大屏显示器展示数据可视化图层。

5.4 本章小结

本章主要是针对本文提出的数据可视化方法以及系统性能测试。首先测试了高分辨率数据可视化图层在大屏展示的效果,对比低分辨率数据可视化图层,验证了为保证可视化效果,低分辨率数据确实会限制屏幕大小,而高分辨率数据可视化图层在大屏展示时效果良好。对于系统性能的测试,主要从系统的通信方式、体系架构、是否采用本文提出的可视化方法进行数据可视化渲染三个方面进行测试。测试结果说明本文系统所使用的通信方式、体系架构对比与传统方式,系统性能都获得了提高;本文提出的数据可视化方法也可以明显提高数据渲染效率,各方面都达到了预期效果。

第六章 总结与展望

未来海洋的战略作用必将越来越重要,国家对于海洋发展的重视程度也必将越来越高。海洋中蕴含着丰富的资源和信息,结合数据可视化技术深入了解海洋,快速掌握海洋变化规律,对我国未来的海洋作业和开发意义重大。本文在国家大力倡导提高海洋开发能力,建设海洋强国的大背景下,针对高分辨率海洋标量数据可视化技术进行研究。基于目前对于海洋标量数据可视化方法的研究现状以及硬件设备研发现状,总结出当前主流的数据可视化方法的局限性。针对这些不足,本文提出了一种基于人眼色差识别阈值的海洋标量数据动态可视化方法,加以应用并对该方法进行优化,基于该方法设计了一套可以用于大屏展示、移动端控制的海洋监测可视化系统。针对系统使用的数据通信技术、确定最终时变阈值采取的调研方法、WebGL 渲染技术、Cesium 框架等进行详细说明与分析,对系统的设计与实现进行了详细的阐述。

本文的重点工作包括针对高分辨率全域数据的快速实时渲染提出数据可视化方法,以及基于该方法设计实现一个海洋高分辨率标量数据实时可视化系统。本文根据人眼色差识别阈值确定时变阈值,基于时变阈值筛选变化数据,在保证可视化效果的前提下提高数据渲染效率。同时,设计时变数据结构,利用时变数据预处理去除冗余数据,利用数据文件更新预处理保证时变数据筛选的正确性,按需更新渲染命令。最后,对可视化方法加以实现,设计完成可视化系统。系统包括数据通信、数据预处理、数据渲染三大模块,实现了图层切换、数据传输与时变数据渲染,提供定点查询、自定义范围划定、要素图表生成等交互功能。系统完成之后,对高低分辨率数据可视化图层在大屏上展示的效果进行对比,对系统的数据通信、处理以及渲染效率进行检测对比,对交互功能完成测试验证。最终,本文提出的数据可视化方法可以提高数据渲染效率,所实现系统的数据通信、处理、传输性能也得到了提高,各项交互功能运行正常。

本文所提出的数据可视化方法主要应用于大屏展示高分辨率海洋标量数据的实时渲染图层,适合于大屏展示、三维球形幕展示以及实时性较高的应用场景。对于后台获取到的最新数据直接渲染展示在大屏上,提高数据渲染的实时性;无需频繁与后台交互请求不同层次、不同经纬度区间的图层,在一定程度上减少了系统网络性能上的消耗;解放了屏幕大小,为大屏、球形幕的数据可视化渲染提供了新的思路。然而,本文所开发设计的系统只支持标量数据的可视化渲染,对于矢量数据的动态可视化渲染将作为后续的研究内容。本文系统是将可视化图层加载到三维地球模型上,严格来说仍是基于二维空间实现数据可视化,实现三维空间下数据高效可视化将是后续研究的重点。

参考文献

- [1] 孙从容, 刘建强, 吴奎桥, 等. 海洋观测卫星地面接收站网的建设和展望[J]. 卫星应用, 2021(09):49-54.
- [2] 王众, 李微微, 杨雯, 等. 海洋环境影响评价及生态修复研究进展与展望[J]. 资源节约与环保, 2021(12):7-10.
- [3] 张彭辉, 杜建平, 吴帅, 等. 海洋环境大数据系统建设研究进展[J]. 海洋开发与管理, 2021, 38(11):104-112.
- [4] 蔡榕硕, 刘克修, 谭红建. 气候变化对中国海洋和海岸带的影响、风险与适应对策[J]. 中国人口·资源与环境, 2020, 30(09):1-8.
- [5] 黄冬梅, 徐宸弋轩, 郑霞, 等. 面向海洋预报任务的大数据可视化系统研究[J]. 计算机工程与应用, 2019, 55(07):87-94.
- [6] 李志炜, 傅军, 韩洪祥. 海洋环境数据三维可视化系统设计与实现[J]. 海洋测绘, 2019, 39(6): 68-72.
- [7] 刘帅, 陈戈, 刘颖洁, 等. 海洋大数据应用技术分析与趋势研究[J]. 中国海洋大学学报(自然科学版), 2020, 50(1): 154-164.
- [8] 郭洁, 张煦, 田春思, 等. 基于数据大屏的综合展示系统的开发与设计[J]. 水电站机电技术, 2021, 44(10):40-42.
- [9] 刘氢. 大屏展示系统的设计与实现[J]. 电子技术与软件工程, 2021(09):43-44.
- [10] LIU X, SHEN W F, LIU B H, et al. Research on Large Screen Visualization Based on Docker[C]// Proceedings of 2018 3rd International Conference on Communication, Image and Signal Processing, 2018: 423-429.
- [11] 王乐谦. 基于 Web 服务的海洋预报数据可视化发布技术研究[D]. 中国石油大学(华东), 2017.
- [12] 程苇杭. 海洋数据可视化平台设计与实现[D]. 浙江海洋大学, 2017.
- [13] 冯剑锋, 王俐纯, 栾英宏, 等. 海洋环境要素探测仪采集处理器的设计与实现[J]. 现代防御技术, 2021, 49(04):79-85.
- [14] 李涛, 张涵. 基于分布式框架的并行瓦片处理技术[J]. 信息化研究, 2021, 47(05):45-48.
- [15] 徐胜华, 刘纪平, 王想红, 等. 基于三维 GIS 的海洋标量场数据动态可视化[J]. 测绘通报, 2013(10):50-53.
- [16] 王思又. 海洋水文数据可视化平台设计与实现[D]. 山东科技大学, 2020.
- [17] CLARK J H. Hierarchical geometric models for visible surface algorithms[J]. Communications of the ACM, 1976, 19(10): 547-554.
- [18] Dunne D, Sutton G. 3D Web Mapping Integrating Marine Data into NASA World Wind [J]. Hydro International, 2007, 10(9): 7-9.
- [19] 朴美燕, 胡毅, 叶迎萍. 基于 Unity 的数字化车间改进资源动态调度算法[J]. 计算机系

- 统应用, 2018, 27(10):196-201.
- [20] Willem H. de Boer. Fast terrain rendering using geometrical mipmapping [OL]. (2000-10) [2012-10-20]. [Http://flipcode.net/archives/article_geomipmaps.pdf](http://flipcode.net/archives/article_geomipmaps.pdf).
- [21] 孔倩倩, 韩勇, 李文庆, 等. 海洋标量数据多维多模式动态可视化系统设计实现[J]. 微计算机信息, 2011, 27(5):177-179.
- [22] 霍超, 刘颖真, 吕蓬, 等. 基于 WorldWind 的三维地理信息系统[J]. 地理空间信息, 2015, 13(05):38-40.
- [23] 孙连华, 刘星. 基于 ArcGIS Explorer 的三维城中村构建研究[J]. 北京测绘, 2013(01):50-54.
- [24] 王晓宇, 孙卡. 基于 osgEarth 的三维虚拟校园可视化[J]. 计算机与现代化, 2020(11):89-93.
- [25] 袁凌, 刘建成, 潘磊, 等. 基于 Cesium 和谷歌影像的风电场地形三维可视化[J]. 地理空间信息, 2020, 18(12): 1-4.
- [26] 王想红, 刘纪平, 王亮, 等. 海洋标量数据的三维动态可视化系统研究[J]. 海洋科学, 2013, 37(7): 90-94.
- [27] 孙强, 王世航, 鹿凤. 网络环境下海洋标量场数据三维可视化研究[J]. 现代电子技术, 2019, 42(8): 104-108.
- [28] 石绥祥, 雷波. 中国数字海洋:理论与实践[M]. 北京:海洋出版社, 2011.
- [29] 张信雪, 吕晓琪, 王月明, 等. 海洋叶绿素 a 浓度动态可视化研究[J]. 海洋科学, 2017, 41(3): 75-79.
- [30] 王琪. 基于 WebGL 的三维数据大屏可视化系统的设计与实现[D]. 北京:北京邮电大学, 2019.
- [31] SHANG Y Y, MENG Y. Single image dehazing on mobile device based on GPU rendering technology[J]. Journal of Robotics, Networking and Artificial Life, 2015, 2(2): 85-88.
- [32] Guo M T, XUE D H, LI P, et al. Vehicle pedestrian detection method based on spatial pyramid pooling and attention mechanism[J]. Information, 2020, 11(12): 583-583.
- [33] 吕建军, 徐仁桐, 郑菊艳. 基于 Unity WebGL 的海洋平台建造虚拟仿真系统[J]. 中国海洋平台, 2021, 36(03):72-77
- [34] 张永强, 王波, 申茂廷. 基于 Cesium 的 3DWebGIS 三维场景加载及开发[J]. 河南科技, 2021, 40(21):8-10.
- [35] 王锦涛. 基于 MQTT 的物联网设备接入及云上监控系统研究[D]. 西安工业大学, 2021.
- [36] 钱鑫博. 基于 WebSocket 服务实现 Web 应用访问本地串口设备[J]. 计算机与网络, 2020, 46(14):62-65.
- [37] 叶康林, 钟瑾瑞, 焦冠文, 等. 基于 MQTT 协议的生产实时数据传输技术应用研究[J]. 信息系统工程, 2021(11):65-68.
- [38] 苏涛. 基于 MQTT 协议的创新型物联网水位监测系统研究[J]. 信息与电脑(理论版),

- 2021, 33(08):130-134.
- [39] 刘禹, 李胜乐, 李力. 大数据平台的测震数据流实时可视化系统设计与实现[J]. 测绘地理信息:1-8[2022-02-21].
- [40] 刘禹. 基于 Websoket 的多源数据流监测平台设计[D]. 中国地震局地震研究所, 2021.
- [41] 孟天杭. 基于 Cesium 的大规模三维场景瓦片构建与可视化[D]. 北京建筑大学, 2021.
- [42] 谢习华, 王刚, 辛涛, 等. 基于 SLIC 和改进区域生长的非结构化道路识别[J/OL]. 计算机工程与应用, 2021:1-11[2021-11-15].
<http://kns.cnki.net/kcms/detail/11.2127.TP.20210327.1437.004.html>.
- [43] 赵小明, 滕鹏超, 宗靖国. 人眼对不同颜色色差辨别能力的研究[J]. 电子科学技术, 2014, 1(3): 303-307.
- [44] 孔月萍. 图像逆半调及其质量评价技术研究[D]. 西安电子科技大学, 2008.
- [45] 罗立鸿. 人眼对比度和色差阈值测试实验研究[D]. 北京理工大学, 2016.
- [46] 张贵强, 王美玲. 基于 NodeJS 的企业网站的设计与实现[J]. 信息技术与信息化, 2019(12):58-60.
- [47] 邱璐璐, 陈俊仁. 基于 SpringBoot 和 Vue 框架的高校代领系统设计与实现[J]. 信息技术与信息化, 2021(04):25-27.
- [48] 夏伟, 艾波, 杨应召, 等. NetCDF 物理海洋数据云存储技术研究[J]. 海洋技术学报, 2019, 38(04):71-78.
- [49] 陈文艺, 张霏, 龙艳. 基于 Socket.IO 的物联网网关实时双向通信系统[J]. 西安邮电大学学报, 2017, 22(06):111-116.
- [50] PIMENTEL V, NICKERSON B G. Communicating and displaying real-time data with WebSocket[J]. IEEE Internet Computing, 2012, 16(4):45-53.
- [51] 王华威. 基于 Node.js 的网上书店设计与实现[J]. 中国新技术新产品, 2020(22):43-46.
- [52] 黄林成. 基于 Node.js 的建筑能耗评估系统的设计[J]. 价值工程, 2020, 39(20):193-194.
- [53] 赵婉琳, 彭仕霖, 杜孟新. 基于 Node.js 的汽车数据中台系统的设计与实现[J]. 仪器仪表标准化与计量, 2021(05):23-26.
- [54] 袁劲松, 马旭东. 基于阻塞与非阻塞 I/O 网络模型的 Java 语言实现[J]. 计算机系统应用, 2008(09):98-101.
- [55] 荣林林. 自适应浏览器的高性能 Web 扩展应用开发系统的研究与设计[D]. 北京邮电大学, 2015.
- [56] 李草. 基于 Node.js+Express 框架的智能燃气监管系统应用研究[D]. 中国石油大学(北京), 2018.
- [57] 崔莹, 刘兵. Node.js 与 Express 技术在计算机课程教学中的应用[J]. 软件导刊, 2016, 15(09):190-192.
- [58] 周信文, 张发勇, 罗津, 等. 基于 Cesium 和 WebGIS 的水文站点信息管理[J]. 水利技术监督, 2021(10):55-59.

- [59] 杨菁, 陈冰凌, 王文鹏, 等. 基于 Cesium 的三维可视化场景建设及发布技术的研究[J]. 测绘通报, 2021(S1):50-53.
- [60] 何强, 李文华, 钟晓峰. 基于 Cesium 的大量倾斜摄影模型加载方法研究[J]. 电脑编程技巧与维护, 2021(07):142-143.

致谢

时光飞逝，三年的研究生时光转瞬即逝。这三年是收获满满的三年，我体会到了科研工作的内在魅力，学会了要一直保持客观公正、诚实守信以及严谨负责的科研态度。

首先，我要感谢张小垒老师。张老师是一位平易近人、学识丰富的优秀教师。是他在这三年中对我的学习任务做了系统性的规划，有计划地安排我的研究任务，耐心解答我的疑问，给予我很大的支持。其次，感谢高鹏翔教授对我科研态度的指导，让我以严谨负责的态度去研究课题。同时，感谢同学和舍友们在学习生活上给予我的帮助。最后，感谢我的父母和朋友，是他们的陪伴和鼓励让我一直前行。

攻读学位期间的研究成果

学术论文:

张广宇, 张小垒. 高分辨率海洋标量场大屏动态可视化方法研究[J]. 青岛大学学报, 2022

申请专利:

[1] 一种基于 ArcGIS 的地下管线探查方法, CN202010206526.0, 中国, 2020.07.10, 张小垒, 郝东升, 张广宇

